

**UNIVERSITÉ JOSEPH KI-ZERBO
(UJKZ)**

**INSTITUT BURKINABÈ DES ARTS ET MÉTIERS
(IBAM)**



**RAPPORT DE STAGE POUR L'OBTENTION DE LA LICENCE
PROFESSIONNELLE**

OPTION : Méthodes Informatiques Appliquées à la Gestion (MIAGE)

Période de stage : 03 juillet 2023 au 03 octobre 2023

THEME :
**MISE EN PLACE D'UN SYSTÈME DE GESTION DE
MISSIONS : CAS DE A2SYS CONSULTING**

Présenté par : KAMBOU Yéri Sheila Mariane

Maitre de stage :
M. SAVADOGO Mahamadi
Chef de projet à A2SYS Consulting

Directeur de rapport :
Dr TRAORE Yaya
Maître de conférences en informatique à
l'IBAM

Année Académique : 2022 - 2023

Sommaire

DEDICACE.....	ii
REMERCIEMENTS	iii
LISTE DES SIGLES ET ABREVIATIONS	iv
LISTE DES FIGURES ET GRAPHIQUES.....	vi
LISTE DES TABLEAUX	viii
INTRODUCTION GENERALE.....	1
CHAPITRE 1 : PRESENTATION DES STRUCTURES DE FORMATION ET D'ACCUEIL	2
I. Présentation de la structure de formation (IBAM)	2
II. Présentation de la structure d'accueil (A2SYS consulting)	6
CHAPITRE 2 : ANALYSE ET CONCEPTION	9
I. Etude préalable.....	9
II. Expression des besoins.....	19
III. Conception globale.....	34
IV. Réalisation.....	43
CHAPITRE 3 : BILAN DU STAGE	61
I. Déroulement du stage et activités menées	61
II. Observations, suggestions	62
CONCLUSION GENERALE	63
BIBLIOGRAPHIE ET WEBOGRAPHIE	I
TABLE DE MATIERES	III
ANNEXES	VI
Annexe 1 : Méthode COCOMO.....	VI
Annexe 2 : Token JWT.....	VII

DEDICACE

À

Toute ma

Famille

REMERCIEMENTS

Nos remerciements vont à l'endroit de toutes les personnes qui ont œuvré de près ou de loin à notre avancée scolaire et estudiantine, tous ceux qui n'ont ménagé aucuns efforts pour l'obtention et le bon déroulement de notre stage et toutes personne ayant participer à l'élaboration de ce document, nos remerciements vont particulièrement à :

- Dieu qui est la pierre angulaire de toutes choses dans notre vie ;
- M. Mahamadi SAVADOGO, notre maitre de stage qui n'a cessé de nous a apporté un soutien multiforme malgré ces occupations ;
- Dr Yaya Traoré, coordonnateur de la filière MIAGE mais également notre maitre de suivi qui a mis en place l'environnement et les ressources nécessaires au bon déroulement de notre travail ;
- Dr Gilbert BAYILI, Directeur de l'IBAM et tout le corps professoral pour les conseils et la formation de qualité que nous avons reçus au cours de ces trois (03) années passées à l'IBAM ;
- tous nos proches et amis qui nous ont toujours soutenu et encouragé.

LISTE DES SIGLES ET ABREVIATIONS

SIGLES ET ABREVIATIONS	SIGNIFICATION
2TUP	Two-Track Unified Process
ABF	Assurance-Banque-Finance
ADB	Assistant de Direction option Bilingue
ADC	Assistant de Direction option Comptable
AGIRH	Gestion des Ressources Humaines en milieu hospitalier
AMOA	Assistant (ou assistance) à maîtrise d'ouvrage
ATOS	Administratifs, Techniques, Ouvriers, de Service
CCA	Comptabilité-Contrôle-Audit
CMMI	Capability Maturity Model Integration
COCOMO	COConstructive COst MOdel
COBIT	Control Objectives for Information and Related Technolog
CSAF	Chef Service Administratif et Financier
ERP	Enterprise Resource Planning
GED	Gestion Electronique de Documents
IBAM	Institut Burkinabè des Arts et Métiers
IBF	Master en Ingénierie Bancaire et Financière
IT	Technologies de l'Information
ITIL	Information Technology Infrastructure Library

M2ISIE	Master en Informatique option Ingénierie des systèmes d'information en entreprise
MAGE	Master en Administration et Gestion des Entreprise
MCCA	Master en Comptabilité-Contrôle-Audit
MIAGE	Méthodes Informatiques Appliquées à la Gestion
MIAGE	Informatique option Méthodes Informatique Appliquées à la Gestion
MID	Marketing et innovation Digitale
RI-MT	Informatique option Réseau Informatique Multi media et Telecom
TIC	Technologies de l'Information et de la Communication
TOGAF	The Open Group Architecture Framework
UEMOA	Union économique et monétaire ouest-africaine
UJKZ	Université Joseph KI-ZERBO
UP	Unified Process

LISTE DES FIGURES ET GRAPHIQUES

Figure 1 : organigramme de l'IBAM	3
Figure 2 : organigramme de A2SYS consulting.....	7
Figure 3 : processus existant de gestion des missions au sein de A2SYS consulting.....	9
Figure 5 : gestion des missions	10
Figure 6 : planning de réalisation.....	19
Figure 7 : diagramme des cas d'utilisation.....	23
Figure 8 : diagrammes de séquences vue de l'extérieur du CU « Authentification »	28
Figure 9 : diagrammes de séquences vue de l'extérieur du CU « Enregistrer une mission » ..	29
Figure 10 : diagrammes de séquences vue de l'extérieur du CU « Gérer les fiches de perdieme »	30
Figure 11 : architecture 2 (Source : Architecture client-serveur (geonov.fr))	32
Figure 12 : architecture 3 (Source : Architecture client-serveur (geonov.fr)).....	33
Figure 14 : diagramme de classe	38
Figure 15 : diagramme d'activité du cas d'utilisation « S'authentifier »	39
Figure 16 : diagramme d'activité du cas d'utilisation « Gérer une mission »	40
Figure 17 : diagramme d'activité du cas d'utilisation « Gérer fiche de perdieme ».....	41
Figure 18 : diagramme de déploiement.....	42
Figure 13 : architecture monolithique (Source : Full stack development with JHipster)	50
Figure 19 : premier résultat des tests fonctionnels.....	52
Figure 20 : deuxième résultat des tests fonctionnels.....	53
Figure : résultats du test fonctionnel	53
Figure 21 : écran page de destination.....	58
Figure 22 : écran de connexion	58
Figure 23 : écran principale.....	59
Figure 24 : ecran d'ajout de missions.....	59

Figure : liste des missions	60
Figure 25 : importation de JUnit	IX
Figure 26 : exemple d'implémentation de test pour le service « save » de la classe mission..	IX
Figure 27 : exemple d'implémentation de test sur la classe mission	X
Figure 28 : détails des fichiers de test fonctionnels	XI

LISTE DES TABLEAUX

Tableau 1 : étude comparative des principales méthodologies de développement	13
Tableau 2 : liste des cas d'utilisation de notre futur système	21
Tableau 3 : description du cas d'utilisation « S'authentifier »	24
Tableau 4 : description du cas d'utilisation « Gérer les missions ».	25
Tableau 5 : description du cas d'utilisation « gestion des fiches de perdieme ».....	26
Tableau 6 : description du cas d'utilisation « gérer les agents »	27
Tableau 7 : description de la classe « Agent »	34
Tableau 8 : description de la classe « Mission »	35
Tableau 9 : description de la classe « Ordre de mission ».....	35
Tableau 10 : description de la classe « Rapport de mission »	35
Tableau 11 : description de la classe « Client ».....	36
Tableau 12 : étude comparative des SGBD.....	43
Tableau 13 : étude comparative des serveurs d'application	46
Tableau 14 : étude comparative des méthodes d'estimation de coût	55
Tableau 15 : coût total du projet.....	57
Tableau 16 : formule de calcul COCOMO.....	VI

INTRODUCTION GENERALE

De manière général, une mission est le fait d'envoyer une ou plusieurs personnes réaliser une tâche bien déterminée. A A2SYS consulting, une mission est constituée d'un ou de plusieurs voyages ou sorties effectués par un ou plusieurs agents au sein ou à l'extérieure du pays dans un but bien précis. La mission se déroule sous la tutelle d'un chef de mission accompagner de son maître d'ouvrage sur une période pouvant varié par agents.

La gestion de ces missions se fait pour la plupart du temps manuellement ou en utilisant des suites bureautiques telle que Office couplé avec un système d'impression. Le cas de A2SYS consulting, ils se sont dotés d'un logiciel qui règle le problème. Mais il reste limité à long terme. Il n'implémente pas tout le processus et est fait via des technologies qui ne sont plus adapter.

C'est fort de ce constat que A2SYS consulting à décider de réaliser une plateforme de gestion de leurs missions, une plateforme qui respectera à la lettre leur processus métier et leurs permettra de gérer tout ce qui est production de document et/ou signature.

C'est dans ce contexte que nous avons été accueillis au sein de ladite entreprise pour un stage de trois (3) mois afin de mettre en place une plateforme répondant à ce besoin. Cette plateforme web devra permettre aux différents acteurs entrant en jeu dans l'établissement d'une mission de pouvoir mener à bien cette tâche.

Pour mener à bien cette étude, notre démarche s'articulera autour de trois axes principaux à savoir :

- ❖ la recherche documentaire,
- ❖ la rédaction du rapport,
- ❖ la réalisation du système.

Le présent document détaille toutes les étapes du processus de réalisation de ce projet. Il est sectionné en trois grandes partie notamment :

- ❖ la présentation des structures de formation et d'accueil,
- ❖ la réalisation d'une plateforme web de gestion de missions,
- ❖ le bilan de notre stage à A2SYS consulting.

CHAPITRE 1 : PRESENTATION DES STRUCTURES DE FORMATION ET D'ACCUEIL

Dans cette partie nous prendrons le temps de vous faire la présentation de notre institut et de notre structure d'accueil.

I. Présentation de la structure de formation (IBAM)

L'Institut Burkinabè des Arts et Métiers (IBAM) est un établissement d'enseignement professionnel de renom au Burkina mais également dans la sous-région.

1. Historique

Sis à Somgandé, l'Institut Burkinabè des Arts et Métiers (IBAM) est un institut d'enseignement professionnel. Il a été créé en janvier 2000 dans le cadre de la refondation de l'Université de Ouagadougou rebaptisée l'Université Ouaga I Pr Joseph KI-ZERBO, le 26 décembre 2015, puis plus simplement « l'Université Joseph KI-ZERBO » depuis le 12 avril 2019. L'IBAM est la matérialisation de l'engagement de l'Université Joseph KI-ZERBO (UJKZ) dans la professionnalisation des filières.

2. Objectif

L'objectif premier de l'IBAM est de répondre aux besoins du marché de l'emploi par la mise à disposition d'un potentiel humain de cadres moyens et supérieurs dans les divers secteurs d'activité. Cette nouvelle orientation se justifie par l'incapacité de l'État d'embaucher tous ceux qui sortent de l'université.

3. Organisation

L'IBAM est organisé en deux organes à savoir l'organe statutaire et l'organe d'exécution. L'organisation des enseignants et l'organisation du contrôle des aptitudes et des connaissances sont fixées par arrêté ministériel. L'institut est dirigé par un Directeur assisté d'un adjoint, tous deux élus par le collège électoral de l'établissement pour un mandat de deux (02) ans renouvelables. La figure 1 présente l'organigramme de l'IBAM, à travers lequel on peut voir les liens hiérarchiques et organisationnels existants entre les personnels de l'IBAM.

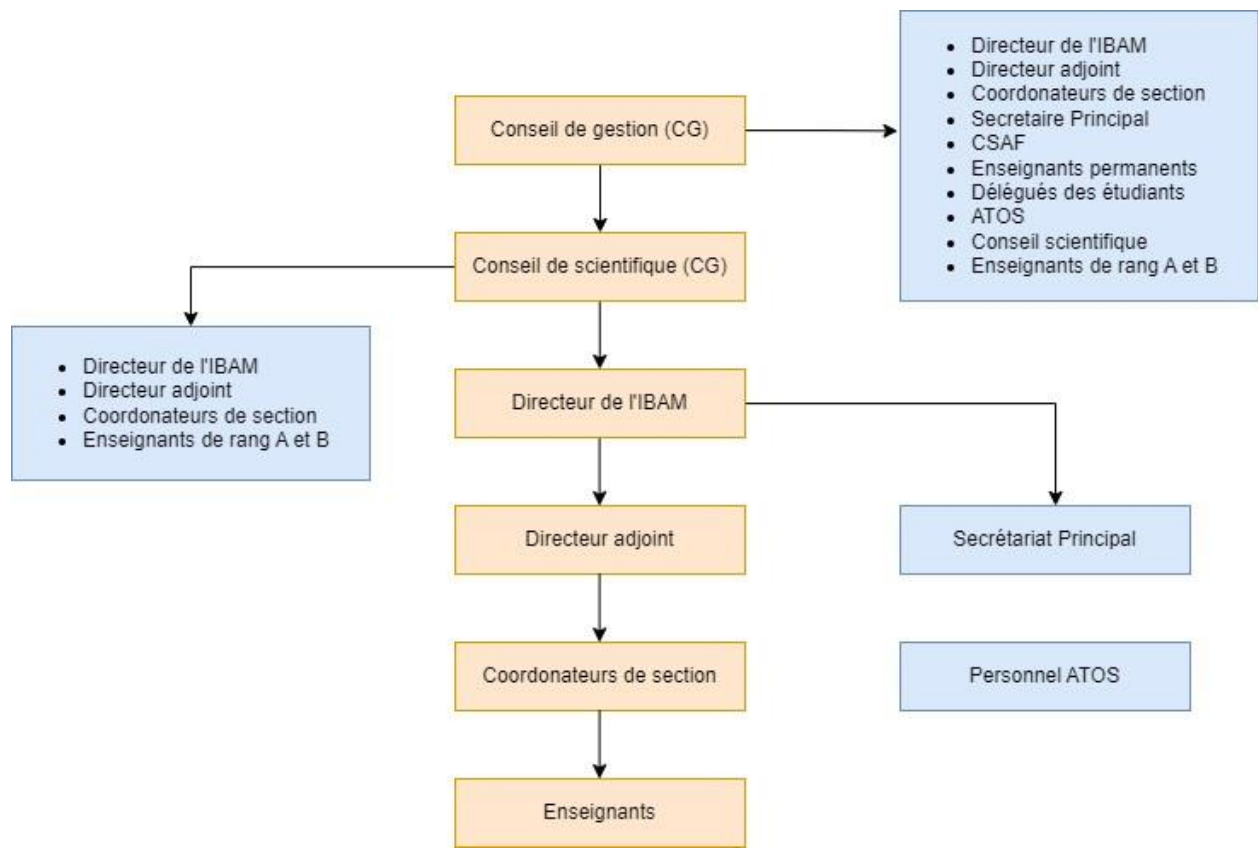


Figure 1 : organigramme de l'IBAM

a. Organe statutaire

L'organe statutaire de l'IBAM est composé du Conseil de gestion et du Conseil scientifique.

Le Conseil de gestion est l'organe qui gère l'institut. Il est composé du Directeur de l'IBAM, du Directeur adjoint, des Coordonateurs de sections, du Secrétaire principal, du Chef de Service Administratif et Financier (CSAF), des Enseignants permanents, de deux (02) représentants des Délégués de syndicats des étudiants et d'un représentant du personnel ATOS. Il est présidé par le Directeur de l'IBAM et se réunit tous les trois (03) mois sauf en cas de session extraordinaire sur convocation de son président. Le Conseil de gestion a pour missions de :

- définir le règlement intérieur de l'institut ;
- proposer le budget de fonctionnement de l'institut ;

- décider des mesures administratives et financières conformément aux textes en vigueur ;
- proposer la modification des statuts de l'IBAM.

Le Conseil scientifique est composé également du Directeur de l'IBAM, du Directeur adjoint, des Coordonnateurs de section, des enseignants de rang A et les enseignants de rang B de l'Institut. Il est présidé par le Directeur de l'IBAM. Il se réunit et délibère dans les mêmes conditions que le Conseil de gestion mais son secrétariat est assuré par un des Coordonnateurs de section. Le Conseil scientifique a pour mission de :

- proposer une organisation générale des enseignements de l'institut ;
- proposer l'ouverture, la fusion ou la fermeture de filières ;
- créer des titres, diplômes et d'examiner les équivalences pédagogiques ;
- proposer la création de nouveaux postes et ;
- apprécier les orientations de la recherche au sein de l'institut.

b. Organe d'exécution

L'organe exécutif a pour mission de mettre en pratique les décisions prises par l'organe statutaire. Il est composé essentiellement de la direction de l'IBAM. La Direction est pilotée par le Directeur de l'IBAM dont les missions sont de diriger, de contrôler le fonctionnement des services et de définir la politique de l'institut. Elle coordonne les activités des différents services et veille au maintien de l'image de marque de l'Institut. En outre, elle veille à la mise en œuvre des décisions, des projets et des programmes de l'Institut. Enfin, elle veille à la mise en application des textes et règlements de l'établissement. Le Directeur de l'IBAM est assisté par une Secrétaire de direction. Rappelons que le Secrétaire principal et le CSAF pour leur part, assistent également le Directeur de l'IBAM dans l'accomplissement de certaines activités en relation avec leurs domaines de spécialisation. La Direction est composée du Secrétariat de direction, du Service administratif et financier et du Secrétariat principal.

4. Filière de formation

En vue d'atteindre les objectifs cités précédemment, l'IBAM offre des formations dans plusieurs filières. Celles-ci sont réparties en deux (02) groupes selon les diplômes, ces groupes sont : le groupe des Licences professionnelles et le groupe des Masters.

❖ **Licences professionnelles**

L'IBAM offre des formations initiales dans la catégorie des licences professionnelles :

- Comptabilité-Contrôle-Audit (CCA) ;
- Assurance-Banque-Finance (ABF) ;
- Assistant de Direction option Bilingue (ADB) ;
- Assistant de Direction option Comptable (ADC) ;
- Informatique option Méthodes Informatique Appliquées à la Gestion (MIAGE) ;
- Informatique option Réseau Informatique Multi media et Telecom (RI-MT) ;
- Marketing et innovation Digitale (MID).

❖ **Masters**

En Master, l'IBAM offre cinq (05) filières de formation qui sont :

- Master en Administration et Gestion des Entreprises (MAGE) ;
- Master en Comptabilité-Contrôle-Audit (MCCA) ;
- Master en Ingénierie Bancaire et Financière (IBF) ;
- Master en Informatique option Ingénierie des systèmes d'information en entreprise (M2ISIE) ;
- Master en informatique option Sécurité Informatique.

La filière MIAGE a été créée dans l'optique de répondre aux besoins croissants des entreprises en cadres compétents dans le domaine des Technologies de l'Information et de la Communication (TIC).

Cette filière accueille en première année les bacheliers des séries C, D et E ayant passé avec succès le test de sélection ou ayant été acceptés en tant qu'auditeurs libres.

Les étudiants en fin de cycle en MIAGE doivent mettre en pratique les connaissances acquises en classe en effectuant un stage d'une durée d'au moins trois (03) mois suivi d'une soutenance publique.

C'est pour répondre à cette exigence académique que nous avons effectué notre stage pratique à A2SYS consulting que nous mettons en avant dans les lignes qui suivent.

II. Présentation de la structure d'accueil (A2SYS consulting)

Dans cette partie, nous présentons la structure dans laquelle nous avons effectué notre stage : A2SYS consulting.

1. Histoire

A2SYS consulting est une société de service informatique, créée en 2010 au Burkina Faso.

Elle a pris son envol en 2017 avec la création des filiales du Mali en 2017 et celle du Niger en 2021. A2SYS consulting est le résultat d'un rêve et d'un fort engagement de jeunes africains passionnés des systèmes d'information, qui ont créé, et ont su faire évoluer leur société qui bénéficie maintenant d'une très bonne notoriété dans la sous-région ouest africaine. Pour rester au plus près de leurs clients, l'entreprise a installé une antenne dans la plupart des capitales de la sous-région ouest africaine.

Cette proximité géographique offre des garanties certaines en termes d'accompagnement, de rapidité d'intervention et de proactivité dans la prise en compte de leurs besoins. Pour renforcer leur offre, elle a établi des partenariats avec des éditeurs à l'échelle mondiale tel que :

- Oracle : un éditeur de système de gestion de base de données relationnelle. Oracle e-Business est une ERP couvrant toutes les activités de gestion de l'entreprise. Le contrat de partenariat date de 2018 ;
- Dbvisit : un éditeur d'une solution de réplication et de mise en œuvre de site de secours Oracle. Le contrat de partenariat date de 2016 ;
- JobRouter : une plateforme complète de digitalisation des services intégrant l'automatisation des processus, des flux de données (Workflow) et la gestion électronique de documents (GED). Le contrat de partenariat date de 2018 ;
- AGIRH : une société spécialisée dans la gestion des ressources humaines et offre, à travers son progiciel AGIRH et ses services, l'ensemble des outils nécessaires pour accompagner les entreprises et institutions dans l'amélioration et l'optimisation de leur capital humain. Le contrat de partenariat date de 2018.

La société a aussi noué de fortes relations avec d'autres intégrateurs africains dans le but d'augmenter sa capacité à répondre à la demande de sa clientèle. Sa démarche s'appuie d'une part, sur la conformité de ses procédures aux normes de qualité et aux références de bonnes pratiques et d'autre part, sur son expérience pratique d'accompagnement des entreprises africaines dans leur transformation numérique.

2. Vision

A2SYS consulting a pour vision d'accroître l'efficacité et la productivité de leurs clients par la mise à disposition de solutions innovantes et performantes, mais également de créer un cadre idéal, accroître le dynamisme collectif avec ses collaborateurs afin qu'ensemble, ils puissent innover et échanger leur quotidien par la technologie. A2SYS consulting aspire à devenir l'un des principaux acteurs du marché des sociétés de service en informatique en Afrique de l'ouest.

3. Organigramme

L'organigramme est une représentation schématique des liens et des relations fonctionnelles, organisationnelles et hiérarchiques qui existent entre les éléments et les individus au sein d'une entreprise.

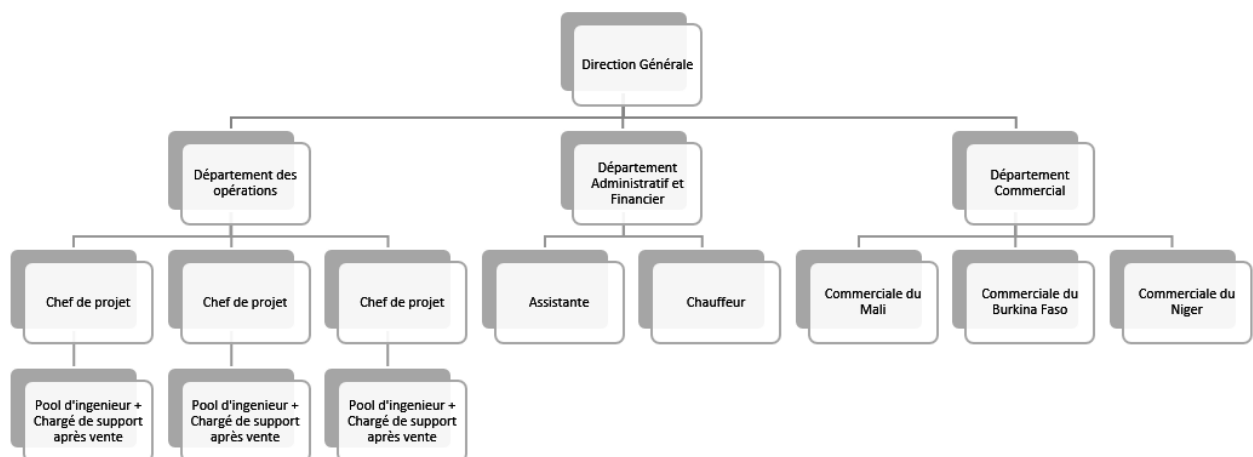


Figure 2 : organigramme de A2SYS consulting

Ceci met en évidence la structure organisationnelle de la société A2SYS consulting. Dans cette organisation il existe un groupe ou une personne se trouve au sommet, tandis que ceux détenant moins de responsabilités se retrouvent en dessous.

Nous avons par exemple le Chef de projet qui commande le pool d'ingénieurs et qui est rattaché au département des opérations relevant de la direction générale.

4. Services

A2SYS consulting propose divers services tels que:

- ❖ **études, conseil et assistance en système d'information** : A2SYS consulting réalise des études qui permettent d'améliorer ou de transformer un système d'information. Ils construisent les trajectoires digitales réalistes et innovante à partir d'un bilan et des projection futures. Les méthodologies utilisées sont conformes aux normes et meilleurs pratiques IT tels que ITIL, COBIT, TOGAF, CMMI et bien d'autres.
- ❖ **intégration de solutions** : A2SYS consulting dispose d'une équipe de proximité capable d'intervenir rapidement dans les huit (08) pays de l'espace UEMOA. Ils agissent également en maître d'œuvre ou en tant que AMOA pour permettre l'implémentation des projets ERP.
- ❖ **développement d'applications** : A2SYS consulting possède une équipe dévouée et pluridisciplinaire capable de conduire de bout en bout différents types de projets. L'équipe utilise une démarche agile pour un meilleur rendu.
- ❖ **digitalisation des processus métiers** : A2SYS consulting est également en mesure d'implémenter tout type de processus métier. Cela permet d'automatiser les processus manuels ou fastidieux qui sont généralement source d'erreurs ou consommation de temps.
- ❖ **renforcement de capacités** : A2SYS consulting possède aussi une salle de formation et des formateur qualifiés et certifiés afin d'offrir des formations de qualités dans les domaines suivants : technologie Oracle et PostgreSQL, l'administration Linux et Unix, la gestion de projet avec MS-Projet, les outils de virtualisation tels que VMware, Oracle VM et Hyper-V, les référentiels de gouvernance informatique tels que ITIL, COBIT et TOGAF et pour finir les environnements de développement open source comme Java, PHP, Laravel.

CHAPITRE 2 : ANALYSE ET CONCEPTION

I. Etude préalable

1. Etude de l'existant

Pour mener à bien notre travail nous passerons premièrement par une étude de l'existant qui consiste à bien comprendre et à décrire le système de gestion de mission existant. Ainsi nous avons pris attache avec le responsable administratif de l'entreprise afin de nous familiariser avec le processus de gestion existant. L'entretien réalisé à cet effet nous permet de décrire le processus de gestion des missions et de définir les limites du système existant.

a. Présentation du processus de gestion des missions

La figure ci-dessous donne une idée sur le processus existant de gestion des missions au sein de A2SYS consulting.

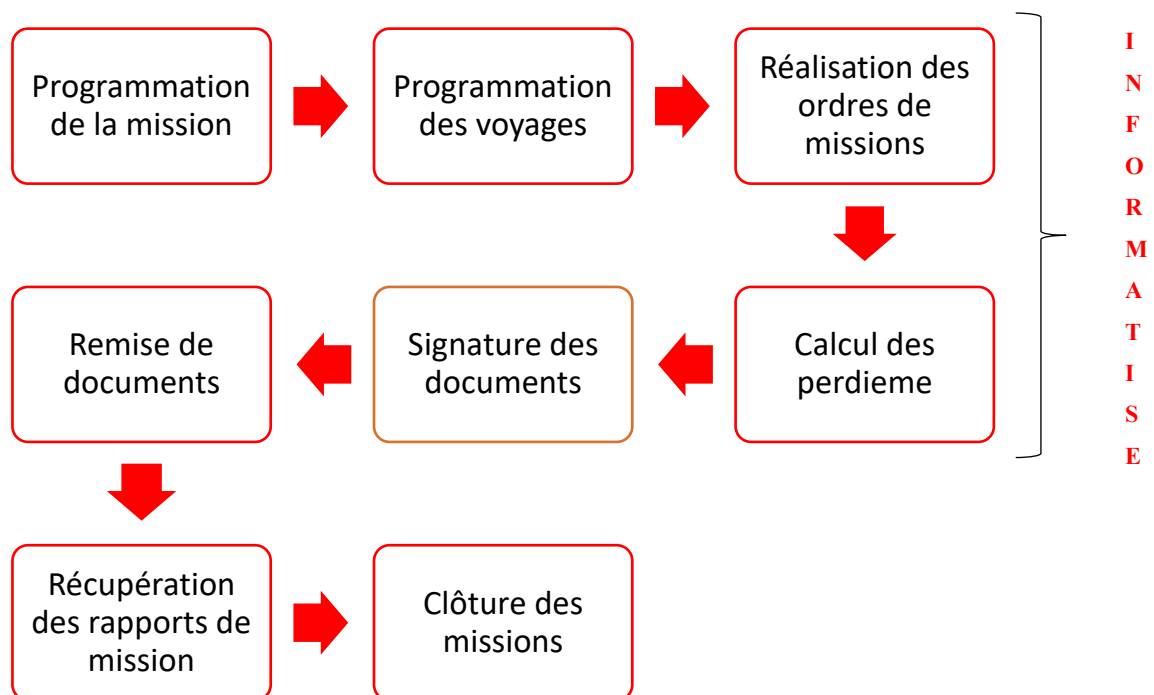


Figure 3 : processus existant de gestion des missions au sein de A2SYS consulting

b. Présentation de l'existant

Le système ci-dessus a été réalisé pour permettre la création des ordres de missions et le calcul des perdiemes, il permet d'entrer des informations prédéfinies et utilise les données choisies afin de générer des ordres de mission et de faire le calcul des perdiemes.

Figure 5 : gestion des missions

c. Limite de l'existant

Ce système est une alternative à la saisie manuelle des informations des ordres de missions, et à l'automatisation du calcul de perdieme. Mais celui-ci possède des limites tel que :

- ❖ Le manque d'automatisation du processus métier dans son intégralité. Nous avons remarqué que le processus métier de A2SYS consulting dans la gestion de ces missions était subdivisé en huit (08) parties comme le stipule la figure 3, ce processus n'est informatisé qu'à 25%.
- ❖ La non centralisation des données : les données tels que les pièces justificatives, les ordres de missions, etc. ne sont pas centralisés et ceux-ci ne sont pas directement liés à leurs missions.
- ❖ Le manque d'accès aux états et statistiques automatique par le gestionnaire : l'application actuelle ne fournit pas de données statistiques, ni d'états de missions, ce qui est nécessaire pour le gestionnaire dans ces comptes.

- ❖ Le manque de lien direct avec les rapports de missions : les rapports de mission sont dissociés de l'application et gérés via un système de cloud. Ce qui rend la gestion globale fastidieuse.
- ❖ Le manque d'accès aux statistiques des agents ainsi que sur le calendrier de mission : les agents n'ont pas directement accès à leur statistique ni à un calendrier prévisionnel.

2. Présentation du thème

La présentation de notre thème s'articulera autour de deux principaux axes à savoir, la problématique puis les attentes.

a. Problématique

A2SYS consulting est soumis à une multitude de missions afin d'assurer leur efficacité auprès de leurs clients. Les missions actuelles sont gérées de façon semi manuelle, ce qui cause une certaine limite au niveau de la digitalisation complète du processus métier. Ce type de gestion révèle sans difficultés les problèmes suivants :

- ❖ le manque d'automatisation du processus métier dans son intégralité : le système actuel étant un système semi manuel et n'intègre pas dans son intégralité le processus métier de la gestion des missions ;
- ❖ le manque d'envoi automatique des ordres de missions : à l'état actuel, les différents documents sont transmis en version physique. On assiste à une impression, une apposition manuelle de signature et pour finir une transmission physique ;
- ❖ la non centralisation des données : les données sur la gestion des missions ne sont pas centralisées, ceux si sont réparties au sein de l'entreprise, les rapports de mission avec les chefs de mission, la gestion des perdiemes avec le gestionnaire, etc. ;
- ❖ le manque d'accès aux états et statistiques automatique par le gestionnaire : les états et statistiques au niveau mission ou au niveau agents ne sont pas disponibles ;
- ❖ le manque de lien direct avec les rapports de missions : la plateforme actuelle ne gère que la production des documents. Elle ne gère pas les rapports de mission, une chose très essentielle à la clôture d'une mission ;
- ❖ le fait que les agents n'aient pas un total accès sur leurs statistiques ainsi que sur le calendrier de mission : les agents n'ont pas d'accès à la plateforme actuelle, seul le gestionnaire peut y interagir.

Tous ces problèmes nous ont conduit à nous demander comment améliorer le système de gestion actuel pour permettre l'automatisation du processus dans son intégralité, améliorer la traçabilité des données et la communication entre les différents acteurs ?

b. Résultats attendus

Au regard des besoins exprimés par l'entreprise, les résultats attendus à l'issue de ce projet sont de mettre en place une application web qui permettra :

- ❖ la gestion des utilisateurs,
- ❖ la gestion des missions,
- ❖ La mise en place de paramètres modifiables,
- ❖ la gestion des voyages,
- ❖ La génération des ordres de missions et des fiches de perdiemes,
- ❖ La gestion des états de missions,
- ❖ La génération des statistiques,

2. Méthodologie

a. Cycle de développement

Le cycle de développement constitue la démarche fondamentale permettant d'obtenir un produit logiciel dans un délai raisonnable tout en minimisant les ressources.

En s'inspirant des rapports (voir bibliographie) nous proposons le tableau suivant qui résume une étude comparative entre les principales méthodologies de développement que nous avons choisi vu la diversité de ces méthodes.

Tableau 1 : étude comparative des principales méthodologies de développement

Méthodologie	Description	Points forts	Points faibles	Coût
Extreme Programming (XP)	XP est une méthodologie de développement logiciel agile qui met l'accent sur la qualité du code, les tests automatisés, la communication étroite entre les développeurs et les clients, ainsi que des itérations courtes pour une réactivité accrue aux changements.	- Adaptation rapide aux changements requis par le client. - Livraisons fréquentes et itératives de fonctionnalités. - Priorisation des tests et de la qualité du code.	- Nécessite un engagement élevé de la part de l'équipe de développement. - Peut être difficile à mettre en œuvre pour les grandes équipes ou les projets complexes.	Le coût de mise en œuvre de XP peut varier selon la taille de l'équipe et le niveau de formation requis, mais il peut être plus élevé en raison de l'accent sur la qualité et les tests.
Two-Track Unified Process (2TUP)	2TUP est une méthodologie de développement logiciel qui combine les pratiques itératives et agiles avec	- Combinaison d'approches agiles et structurées. - Possibilité de gérer des projets plus grands et plus complexes.	- La combinaison de deux approches peut rendre le processus plus complexe. - Nécessite une bonne compréhension des deux	Le coût de mise en œuvre de 2TUP peut être modéré en raison de l'équilibre entre les pratiques agiles et UP.

	les aspects plus planifiés et structurés du processus unifié (UP).	- Flexibilité tout en maintenant une certaine discipline.	approches pour une mise en œuvre efficace.	
Scrum	Scrum est une méthode agile populaire qui se concentre sur la collaboration, la livraison itérative de fonctionnalités, les revues régulières et la planification basée sur les objectifs à court terme (sprints).	- Communication et collaboration étroites entre l'équipe et le client. - Flexibilité pour s'adapter aux changements de priorités. - Livraisons fréquentes et incrémentielles.	- Peut être difficile à mettre en œuvre pour des équipes non familières avec les méthodes agiles. - Nécessite une implication continue du client et de l'équipe.	Le coût de mise en œuvre de Scrum est généralement modéré, avec un investissement initial dans la formation de l'équipe.
Cascade (Waterfall)	Le modèle en cascade est une approche linéaire et séquentielle du développement logiciel, où chaque phase doit être	- Structure et clarté dans le processus de développement. - Facile à comprendre et à gérer pour des projets simples.	- Peu d'adaptabilité aux changements en cours de projet. - Les retours des clients sont limités jusqu'à la fin du projet.	Le coût de mise en œuvre de la cascade peut être relativement bas, mais les coûts de correction d'erreurs tardives peuvent être élevés.

	complétée avant de passer à la suivante.			
Rapid Prototyping (RP)	Le prototypage rapide consiste à créer rapidement des prototypes fonctionnels pour valider les concepts et les fonctionnalités du produit.	- Validation rapide des idées et des concepts. - Réduction des risques en identifiant rapidement les problèmes potentiels.	- Peut conduire à des approches fragmentées si le prototypage n'est pas suivi d'un processus de développement formel. - La qualité du code et la gestion des fonctionnalités peuvent être compromises si les prototypes ne sont pas bien gérés.	Le coût de mise en œuvre du prototypage rapide peut être modéré, avec des avantages potentiels en termes de réduction des risques et des coûts de développement ultérieurs.

Cette étude nous permet de tirer les conclusions suivantes :

- ❖ XP est plus utilisé pour les projets de développement logiciel où les exigences sont susceptibles de changer fréquemment et où l'adaptation rapide est cruciale ;
- ❖ 2TUP est utilisé dans des projets où une approche flexible et itérative est nécessaire tout en conservant une certaine structure et une planification initiale ;
- ❖ Scrum est largement utilisé dans les projets logiciels, mais il peut également être adapté à d'autres domaines ;
- ❖ La cascade est souvent utilisée dans des projets avec des exigences bien définies et stables ;
- ❖ Rapid Prototyping est utilisé pour explorer rapidement de nouvelles idées et concept, généralement en début de projet.

Après cette étude, notre choix s'est porté sur le 2TUP. Au niveau budget, il répond aux capacités de l'entreprise. Le groupe de projet se limite à nous même ce qui facilite son application et permet de séparer les contraintes fonctionnelles des contraintes techniques. Il répond donc parfaitement à nos besoins actuels et nous permettra d'atteindre nos objectifs cités précédemment.

Le 2TUP s'articule principalement autour de deux concepts, le fonctionnel et la technique. Sa forme en Y est un rappel que les besoins fonctionnels et les besoins techniques sont à assembler afin de procéder à la réalisation dudit projet. Cette méthode s'articule autour de deux principaux points :

- ❖ Branche technique :
 - la capture des besoins techniques qui recense toutes les contraintes et les choix dimensionnant la conception du système. Les outils et les matériels sélectionnés ainsi que la prise en compte de contraintes d'intégration avec l'existant conditionnent généralement des prérequis d'architecture technique ;
 - la conception générique, qui définit ensuite les composants nécessaires à la construction de l'architecture technique. Cette conception est complètement indépendante des aspects fonctionnels. Elle a pour objectif d'uniformiser et de réutiliser les mêmes mécanismes pour tout un système. L'architecture technique construit le squelette du système informatique et écarte la plupart des risques de niveau technique. L'importance de sa réussite est qu'il est conseillé de réaliser un prototype pour assurer sa validité.

❖ Branche fonctionnelle :

- la capture des besoins fonctionnels qui produit un modèle des besoins focalisés sur le métier des utilisateurs. Elle qualifie au plus tôt le risque de produire un système inadapté aux utilisateurs. De son côté, la maîtrise d'œuvre consolide les spécifications et en vérifie la cohérence et l'exhaustivité ;
- l'analyse qui consiste à étudier précisément la spécification fonctionnelle de manière à obtenir une idée de ce que va réaliser le système en termes de métier.

Les résultats de l'analyse ne dépendent d'aucune technologie particulière.

❖ Partie réalisation :

- la conception préliminaire, qui représente une étape délicate, car elle intègre le modèle d'analyse dans l'architecture technique de manière à tracer la cartographie des composants du système à développer ;
- la conception détaillée, qui étudie ensuite comment réaliser chaque composant ;
- l'étape de codage, qui produit ces composants et teste au fur et à mesure les unités de code réalisées ;
- l'étape de recette, qui consiste enfin à valider les fonctions du système développé.

Le processus 2TUP s'appuie sur UML tout au long du cycle de développement, car les différents diagrammes de ce dernier permettent en plus de leur facilité et de leur clarté, de bien modéliser le système à chaque étape.

b. Langage de modélisation

Un langage de modélisation est un langage artificiel qui peut être utilisé pour exprimer de l'information ou de la connaissance des systèmes dans une structure qui est définie par un ensemble cohérent de règles. Pour développer une application, il faut d'abord organiser les idées, les documenter avant de commencer la réalisation tout en définissant les modules et les étapes. On appelle cette démarche la « modélisation ». Pour réaliser cette modélisation, nous avons utilisé le langage UML pour plusieurs raisons. Nous pouvons citer entre autres qu'il :

- ❖ présente l'avantage d'être le standard en matière de modélisation objet universellement reconnue ;
- ❖ est un langage visuel car sa notation graphique permet d'exprimer visuellement des solutions objets facilitant ainsi la comparaison et l'évaluation de celles-ci ;

- ❖ est un langage formel et normalisé doté d'un gain de précision et d'un gage de stabilité ;
- ❖ sert à formaliser tous les documents techniques d'un projet et permet d'affiner les détails de l'analyse au fur et à mesure de l'avancée du projet ;
- ❖ est capable d'utiliser le même atelier de génie logiciel, depuis l'expression des besoins des utilisateurs jusqu'à la génération de tout ou partie du code ;
- ❖ est un support de communication performant car, il cadre l'analyse tout en facilitant la compréhension des représentations abstraites complexes.

3. Groupe de travail

a. Groupe de pilotage

C'est le groupe dirigeant chargé de veiller à la bonne mise en œuvre du projet. Il a à sa charge la planification des dates clés du projet, l'examen des propositions du groupe de projet et décide des orientations optimales. Ce groupe est composé de :

- M. SAVADOGO Mahamadi, notre maître de stage,
- M. SANA, notre personne ressource au sein de l'entreprise,
- M. TRAORE Yaya, notre directeur de rapport.

b. Groupe de projet

Le groupe de projet est l'ensemble des personnes chargées de réaliser le projet. Il est l'intermédiaire entre le groupe de pilotage et le groupe des utilisateurs. Ce groupe est composé principalement de Yeri Sheila Mariane KAMBOU, étudiante en 3ème année en Méthodes Informatiques Appliquées à la Gestion (MIAGE) à l'IBAM.

c. Groupe des utilisateurs

Ce groupe est constitué de l'ensemble de tous ceux qui vont utiliser le futur système. Il participe à la capture des besoins fonctionnels. Il est composé principalement des employés de A2SYS consulting.

d. Planning de réalisation

Ce planning permet de programmer les opérations à mettre en place pour la réalisation du projet. Il donne la date, la durée et l'avancement des différentes opérations.

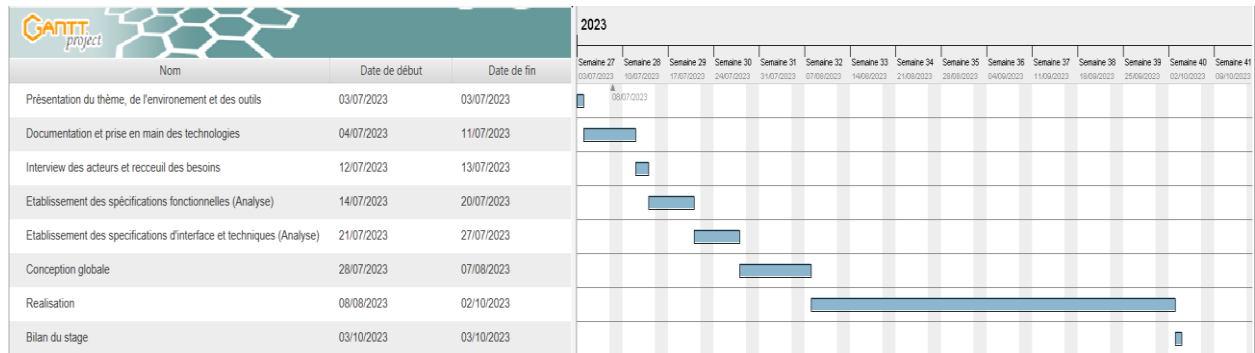


Figure 6 : planning de réalisation

II. Expression des besoins

1. Description du processus de fonctionnement attendu de l'application

L'application que nous allons mettre en place devra permettre à A2SYS consulting de gérer des missions dans tout son processus. Elle devra également répondre parfaitement au processus métier de l'entreprise.

Celle-ci comportera un administrateur principal qui sera chargé de créer les comptes des agents et de leur transmettre leurs identifiants, il sera également chargé de gérer le paramétrage complexe de l'application telle que l'attribution de rôles. Elle permettra de gérer les créations de missions tout en générant les documents administratifs nécessaires et leur envoi, de voir les missions programmées et validé les fiches qui seront envoyés.

L'application permettra de gérer la création de missions, l'attribution d'agents à une mission, le paramétrage simple de l'application, la création de missions, l'allocation des perdiem, la gestion des fiches de perdiem, l'activation/désactivation des paramètres, mais également de l'ajout de pièces justificatives si besoin. Nous créerons également la possibilité de voir les missions, de joindre les rapports de missions, de signer les fiches de perdiem, d'avoir accès aux états des missions et aux statistiques des agents.

2. Spécification fonctionnelle

Les spécifications fonctionnelles font la description des fonctions d'un logiciel en vue de sa réalisation. Dans cette partie, nous décrivons dans les détails les exigences du système à travers l'identification des utilisateurs finaux, des cas d'utilisation et des différents diagrammes.

a. Identification des acteurs

Un acteur est une entité qui définit le rôle joué par un utilisateur ou par un système qui interagit avec le système modélisé. Les acteurs apparaissent dans les diagrammes de cas d'utilisation. Dans notre cas, les utilisateurs seront :

- ❖ admin : qui supervisera la plateforme ;
- ❖ les responsables administratifs : charger de signer l'ordre de missions, générer les fiches de perdieme, et fournir les pièces justificatives ;
- ❖ chefs de missions : il s'agit d'un agent lambda qui est en plus chargé de l'amendement et la validation des rapports de missions ;
- ❖ agents lambda accès en lecture des missions (calendrier de missions), de leurs profils et statistiques et validation des fiches de perdiemes.

b. Identification des cas d'utilisation

Les cas d'utilisations désignent l'ensemble des interactions qui vont permettre à l'acteur d'atteindre son objectif en utilisant le système. Le tableau suivant liste les cas d'utilisation du futur système.

Tableau 2 : liste des cas d'utilisation de notre futur système

Numéro	Cas d'utilisation	Description	Acteurs
CU01	S'authentifier	L'utilisateur arrivant sur la plateforme doit se connecter avant d'accéder au tableau de bord.	Tous les acteurs
CU02	Gérer les missions	Créer, mettre à jour, supprimer.	Responsable administratif
CU03	Gérer les voyages	Créer, mettre à jour, supprimer	Responsable administratif
CU04	Générer les ordres de missions	Créer, mettre à jour, supprimer	Responsable administratif
CU05	Générer les fiches de perdieme	Envoyer les résumer des fiches de perdieme, générer les PDF	Responsable administratif
CU06	Consulter et signer les fiches de perdieme	Vérifier et valider les fichier de perdieme	Agents lambda, Chefs de missions
CU07	Joindre les documents justificatifs	Insérer les fichiers justificatif	Responsable administratif
CU08	Gérer les pièces justificatives	Insérer ou créer tous documents relatif à une justification monétaire	Responsable administratif
CU09	Envoyer les ordres de missions	Envoyer les ordres de missions aux agents	Responsable administratif

CU10	Gérer les rapports de missions	Joindre les rapports Amender/valider les rapports	Chefs de missions
CU11	Consulter les ordres de missions	Lire les ordres de missions	Responsable administratif, Agents lambda, Chefs de missions
CU12	Consulter les statistiques	Lire les statistiques générer	Tous les acteurs
CU13	Gérer les agents	Activer les comptes Envoyer les identifiants aux agents	Admin
CU14	Gérer le paramétrage	Ajouter modifier et désactiver les paramètres	Admin, Responsable administratif

c. Diagramme de cas d'utilisation

Le diagramme des cas d'utilisation est un diagramme UML utilisé pour donner une vision globale du comportement fonctionnel d'un système logiciel. Il permet d'une part de modéliser les besoins des utilisateurs en les clarifiant et en les organisant et d'autre part d'identifier les acteurs et les fonctionnalités du système comme le montre la figure ci-dessous :

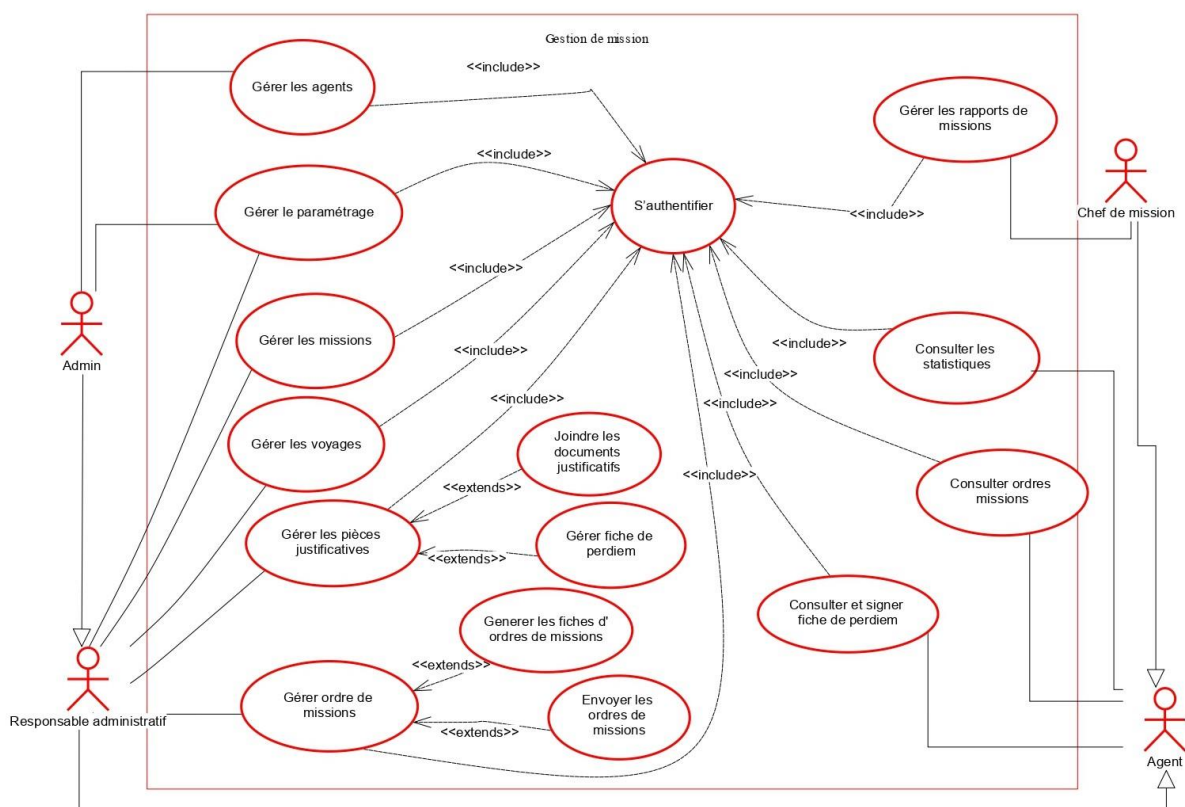


Figure 7 : diagramme des cas d'utilisation

d. Description textuelle de certains cas d'utilisation

Le diagramme de cas d'utilisation décrit les grandes fonctions du système du point de vue des acteurs, la description textuelle quant à elle permet d'exposer de façon détaillée le dialogue entre les acteurs et le système. Pour faciliter la lecture de notre rapport nous nous limiterons à la description de quatre (4) cas d'utilisation. Les cas d'utilisation qui sont décrits dans cette partie sont : s'authentifier, gérer les missions, gérer les fiches de perdieme, gérer les agents.

❖ **S'authentifier**

Le tableau 3 décrit les interactions entre les acteurs et le système pour le cas d'utilisation « S'authentifier ». D'abord, l'utilisateur demande à se connecter, puis il reçoit un formulaire de connexion qu'il doit remplir en fournissant son adresse électronique et son mot de passe. Une fois soumis, le système vérifie les informations saisies. Si les informations sont incorrectes le système lui notifie et renvoie le même formulaire pour qu'il réessaie. Au cas où les informations sont correctes l'utilisateur est dirigé vers la page d'accueil qui le concerne. La procédure est valable pour tous les acteurs du système.

Tableau 3 : description du cas d'utilisation « S'authentifier »

Résumé	Ce cas permet à l'utilisateur de s'authentifier
Acteurs	Tous les acteurs
Préconditions	Néant
Scénario nominal	1. L'acteur demande à se connecter 2. Le système l'affiche un formulaire de connexion 3. L'acteur remplit le formulaire et soumet au système 4. Le système vérifie les données saisies [A] 5. Le système redirige l'acteur vers la page principale
Scénario alternatif	[A] : Identifiants ou mot de passe incorrect 1. Le système notifie l'acteur de l'échec de la connexion 2. Le système renvoie le formulaire de connexion et le scénario reprend à partir du point 3 du scénario nominal.
Post condition	L'acteur accède à son tableau de bord

❖ **Gérer les missions**

Le tableau 4 décrit les interactions entre les acteurs et le système pour le cas d'utilisation « Gérer les missions ». D'abord, l'utilisateur demande à créer une mission, il reçoit donc un

formulaire à remplir avec les champs affiliés à une mission. Une fois soumis, le système vérifie les informations saisies. Si les informations sont incorrectes le système lui renvoie le même formulaire pour qu'il réessaie. Au cas où les informations sont correctes l'utilisateur est redirigé vers la liste des missions.

Tableau 4 : description du cas d'utilisation « Gérer les missions ».

Résumé	Ce cas permet à l'utilisateur de créer une mission
Acteurs	Responsable administratif
Préconditions	S'authentifier
Scénario nominal	1. L'acteur demande à créer une mission 2. Le système l'affiche un formulaire de création 3. L'acteur remplit le formulaire et soumet au système 4. Le système vérifie les données [A] 7. Le système redirige l'acteur vers la liste des missions
Scénario alternatif	[A] : Informations remplies incorrectes 1. Le système notifie l'acteur de l'erreur faite dans le formulaire 2. Le système renvoi le formulaire et le scénario reprend à partir du point 3 du scénario nominal.
Post condition	Le système enregistre la mission et notifie l'acteur de la création avec succès

❖ Gérer les fiches de perdieme

Pour ce cas, nous avons la gestion des fiches de perdieme. Après la création d'une mission, la fiche de perdieme est générée. Le résumé de cette fiche est transmis aux agents concernés qui doivent valider. Après la validation, leur signature y est apposée et une notification est envoyée au responsable administratif pour confirmer la validation de la fiche. Dans le cas où la fiche n'est pas validée, un commentaire est envoyé au responsable administratif pour explication.

Tableau 5 : description du cas d'utilisation « gestion des fiches de perdieme »

Résumé	Ce cas permet à l'utilisateur de gérer les fiches de perdieme
Acteurs	Responsable administratif, agents lambda, chef de mission
Préconditions	S'authentifier, gérer les missions
Scénario nominal	1. Le responsable administratif après avoir renseigné les perdiemes lors de la création de la mission, demande à envoyer les perdiemes aux agents 2. Le système transfère ce résumé aux agents 3. Les agents accèdent au résumer et les valident [A] 4. Le système appose les signatures des agents et renvoie le pdf généré au responsable administratif
Scénario alternatif	[A] : L'agent ne valide pas les perdiemes 1. Le système lui envoie un champ commentaire à remplir 2. Le commentaire est envoyé au responsable administratif 3. Le responsable administratif voit le commentaire fait les modifications et le scénario reprend à partir du point 1
Post condition	Le système génère la fiche signée et notifie le responsable administratif de la validation

❖ Gérer les agents

Dans ce cas, l'utilisateur demande à créer un agent, il reçoit un formulaire à remplir avec les champs affiliés à un agent. Une fois soumis, le système vérifie les informations saisies. Si les informations sont incorrectes le système lui renvoie le même formulaire pour qu'il réessaie. Au cas où les informations sont correctes l'utilisateur est redirigé vers la liste des agents.

Tableau 6 : description du cas d'utilisation « gérer les agents »

Résumé	Ce cas permet à l'utilisateur de créer une mission
Acteurs	Admin
Préconditions	S'authentifier
Scénario nominal	1. L'acteur demande à créer un agent 2. Le système l'affiche un formulaire de création 3. L'acteur remplit le formulaire et soumet au système 4. Le système vérifie les données [A] 7. Le système redirige l'acteur vers la liste des agents
Scénario alternatif	[A] : Informations remplies incorrectes 1. Le système notifie l'acteur de l'erreur faite dans le formulaire 2. Le système renvoi le formulaire et le scénario reprend à partir du point 3 du scénario nominal.
Post condition	Le système enregistre l'agent et notifie l'acteur de la création avec succès

e. Diagramme de séquence

Les diagrammes de séquences représentent les interactions entre le système et les utilisateurs en montrant sous forme de scénarios la chronologie des envois de messages issus d'un cas d'utilisation donné. Les diagrammes de séquences qui sont décrits dans cette partie sont : s'authentifier, gérer les missions, gérer les fiches de perdieme, gérer les agents.

Les flèches orientées vers la droite présentent les messages de demande effectués par les acteurs et celles orientées vers la gauche présentent les messages de réponses renvoyés par le système. Les lignes verticales présentent les temps d'activités des acteurs et la lecture se fait du haut en bas.

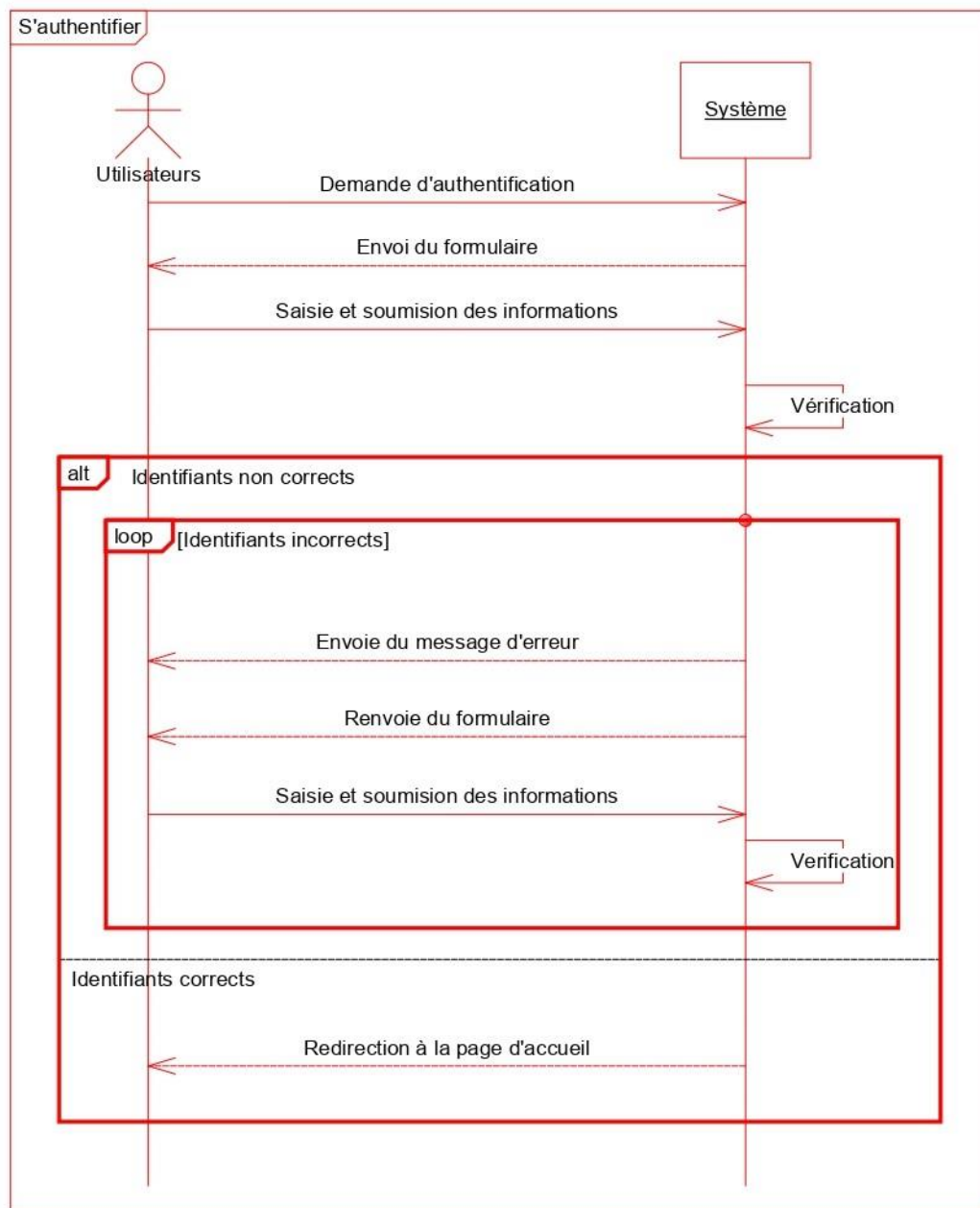


Figure 8 : diagrammes de séquences vue de l'extérieur du CU « Authentification »

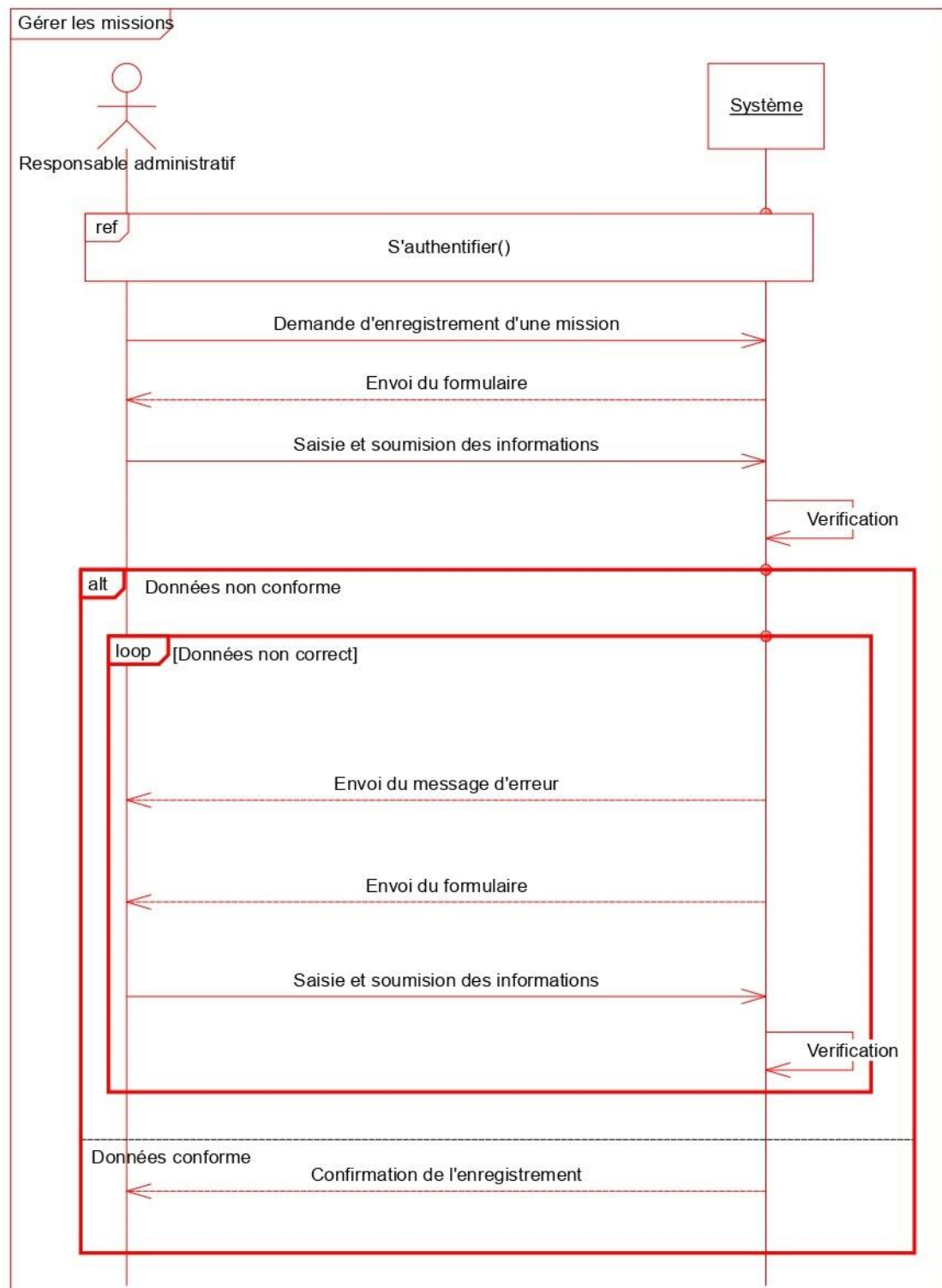


Figure 9 : diagrammes de séquences vue de l'extérieur du CU « Enregistrer une mission »

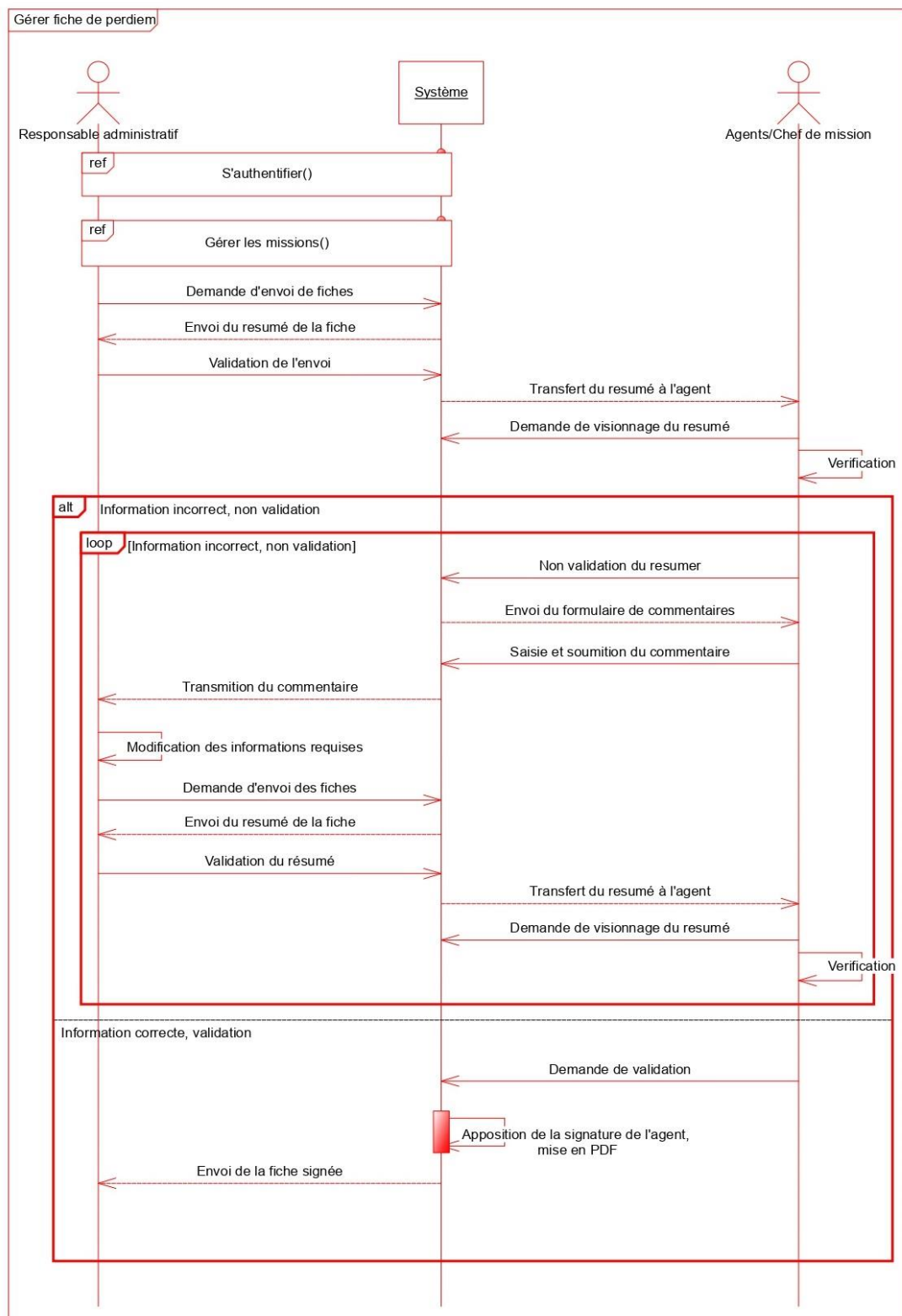


Figure 10 : diagrammes de séquences vue de l'extérieur du CU « Gérer les fiches de perdieme »

3. Spécifications techniques

La spécification technique est la description des choix techniques pris pour satisfaire les spécifications fonctionnelles et les besoins.

a. Mise à disposition des conditions de travail

A2SYS consulting a mis à notre disposition un serveur et une connexion internet haut débit pour nos travaux.

b. Architecture de développement

L'architecture en informatique fait référence à la communication à travers un réseau entre plusieurs éléments physiques et/ou logiques. Il en existe plusieurs types mais nous nous attarderons principalement sur les architectures, deux (2) tiers et trois (3) tiers.

➤ L'architecture deux (2) tiers

Cette architecture, aussi appelée architecture client/serveur de première génération, est constituée de deux niveaux :

- **un client** qui gère la présentation et la logique applicative ;
- **un serveur** qui stocke les données de façon cohérente et éventuellement une partie de la logique applicative.

La communication entre ces deux parties se fait en utilisant le langage SQL. Dans ce type d'architecture, on constate une certaine indépendance du client par rapport au serveur. Dans ce contexte, l'application devra être installée sur tous les postes clients c'est-à-dire sur toutes les machines des différents acteurs du système. Quant à la base de données, elle sera sur un serveur de données centralisé. La figure ci-dessous représente cette architecture.

• Avantages

L'architecture deux (2) tiers présente les avantages suivants :

- elle permet une exploitation multi utilisateur de l'application : plusieurs utilisateurs peuvent avoir accès à la même base de données ;
- elle répartit les charges entre les machines : le client s'occupe de l'interface graphique ainsi que les traitements sur les données tandis que le serveur s'occupe de la recherche des données pour satisfaire aux requêtes des clients ;

- elle offre une bonne sécurité des données : la sécurité des données est assurée au niveau du système de gestion de la base de données.

- **Inconvénients**

L'architecture deux (2) tiers présente les inconvénients suivants :

- le coût de déploiement élevé : il faut installer l'application sur tous les postes clients et les configurer
- l'évolution difficile du système : à chaque mise à jour, de l'application correspond à un redéploiement sur tous les postes clients, ce qui entraîne de nouveaux coûts.

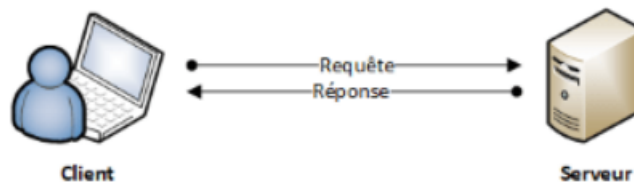


Figure 11 : architecture 2 (Source : [Architecture client-serveur \(geonov.fr\)](http://geonov.fr))

➤ **L'architecture trois (3) tiers**

L'architecture trois (3) tiers est une architecture d'application logicielle bien établie qui organise les applications en trois niveaux de calcul logiques et physiques :

- **le client léger** : encore appelé niveau présentation, est l'interface utilisateur et la couche de communication de l'application, où l'utilisateur final interagit avec l'application. Sa principale fonction est d'afficher des informations à l'attention de l'utilisateur et d'en collecter de ce dernier. Ce niveau supérieur peut s'exécuter sur un navigateur Web, en tant qu'application pour ordinateur de bureau, ou interface graphique utilisateur par exemple ;
- **le serveur d'application** ou niveau application est également appelé niveau logique ou niveau intermédiaire. Il est le cœur de l'application. Dans ce niveau, les informations collectées dans le niveau présentation sont traitées via les informations dans le niveau données en utilisant la logique applicative ;
- **le serveur de base de données** ou niveau données, parfois appelé système dorsal, est l'endroit où les informations traitées par l'application sont stockées et gérées.

Cette méthode comporte tout de même des avantages et des inconvénients :

- **Avantages**

Cette architecture se développe très vite grâce à ses multiples avantages que sont :

- une bonne disponibilité et une évolution facile : les applications trois (3) tiers sont caractérisées par une facilité d'exploitation et une grande flexibilité pour l'évolution. En effet, les mises à jour sont faites au niveau du serveur d'application, ce qui n'entraîne pas le redéploiement sur les postes clients ;
- un déploiement aisé : l'application n'est déployée que sur le serveur d'application. Les clients n'ont besoin que d'un navigateur web compatible avec l'application ;
- une sécurité accrue : ces types d'application possèdent une grande sécurité des données. En effet, l'accès à la base de données est effectué uniquement par le serveur d'application contrairement aux applications deux tiers où tous les utilisateurs sont connectés à la base. Il suffit donc de gérer la sécurité au niveau du serveur d'application ;
- une meilleure gestion de la montée en charge : les instances serveur applicatives peuvent être multipliées pour gérer des montées en charges en mettant en amont un « load balancer ».

- **Inconvénients**

Cette technologie est nouvelle et nécessite un personnel informatique initié pour sa mise en œuvre. De plus, il est recommandé d'exploiter ce type d'architecture dans un réseau haut débit.



Figure 12 : architecture 3 (Source : [Architecture client-serveur \(geonov.fr\)](http://Architecture client-serveur (geonov.fr)))

Solution retenue : en effet notre choix se porte sur l'architecture trois (3) tiers. Elle est conforme aux standards d'architectures d'applications utilisées par A2SYS-Consulting et possède une bonne disponibilité et une évolution facile.

III. Conception globale

1. Dictionnaire de données

Le dictionnaire de données contient l'ensemble des descriptions des attributs de chaque classe qui est utilisé dans le système. Pour faciliter la lecture, nous allons faire la description classe par classe et nous décrirons essentiellement les classes principales.

➤ Agent

Tableau 7 : description de la classe « Agent »

Attribut	Description	Type de donnée
idAgent	Identifiant de l'agent	Nombre
nomAgent	Nom de l'agent	Chaine de caractères
prenomAgent	Prénom de l'agent	Chaine de caractères
dateNaissance	Date de naissance de l'agent	Date
email	Email de l'agent	Chaine de caractères
numCnib	Numéro Cnib de l'agent	Chaine de caractères
contact	Contact de l'agent	Chaine de caractères
signature	Signature de l'agent	Chaine de caractères
imageUrl	Phot de l'agent	Chaine de caractères

➤ **Mission****Tableau 8 : description de la classe « Mission »**

Attribut	Description	Type de donnée
idMission	Identifiant de la mission	Nombre
Code	Code de la mission	Chaine de caractères
Objet	Objet de la mission	Chaine de caractères
description	Description de la mission	Chaine de caractères
Durée	Durée de la mission	Entier

➤ **Ordre de mission****Tableau 9 : description de la classe « Ordre de mission »**

Attribut	Description	Type de donnée
idOrdre	Identifiant de l'ordre de mission	Nombre
url	L'url de l'ordre de mission	Chaine de caractères

➤ **Rapport de mission****Tableau 10 : description de la classe « Rapport de mission »**

Attribut	Description	Type de donnée
idRapport	Identifiant du rapport de mission	Nombre
url	Url du rapport de mission	Chaine de caractères
auteur	Auteur du rapport de mission	Chaine de caractères
version	Version du rapport de mission	Chaine de caractères
commentaire	Commentaire du rapport de mission	Chaine de caractères

Date	Date de dépôt du rapport de mission	Date
Type	Type de rapport de mission	Chaine de caractères
Objet	Objet du rapport de mission	Chaine de caractères

➤ **Client**

Tableau 11 : description de la classe « Client »

Attribut	Description	Type de donnée
idClient	Identifiant du client	Nombre
aisonSociale	Nom du client	Chaine de caractères
address	Adresse du client	Chaine de caractères
telephone	Numéro du client	Chaine de caractères
Email	Email du client	Chaine de caractères
Sigle	Sigle du client	Chaine de caractères
codeClient	Code du client	Chaine de caractères

2. Diagramme de classe

Un diagramme de classe décrit clairement la structure d'un système particulier en modélisant ses classes, ses attributs, ses opérations et les relations entre ses objets.

Les diagrammes de classes présentent de nombreux avantages pour n'importe quel type d'organisation. Ils peuvent être utilisés pour :

- illustrer des modèles de données pour des systèmes d'information, quel que soit leur degré de complexité ;
- mieux comprendre l'aperçu général des schémas d'une application ;
- exprimer visuellement les besoins d'un système et diffuser cette information dans toute l'entreprise ;

- créer des schémas détaillés qui mettent l'accent sur le code spécifique qui doit être programmé et mis en œuvre dans la structure décrite ;
- fournir une description indépendante de l'implémentation des types utilisés dans un système, qui sont ensuite transmis entre ses composants.

Notre diagramme de classe est composé de 19 classes. Elles disposent toutes des attributs définis dans notre dictionnaire de données.

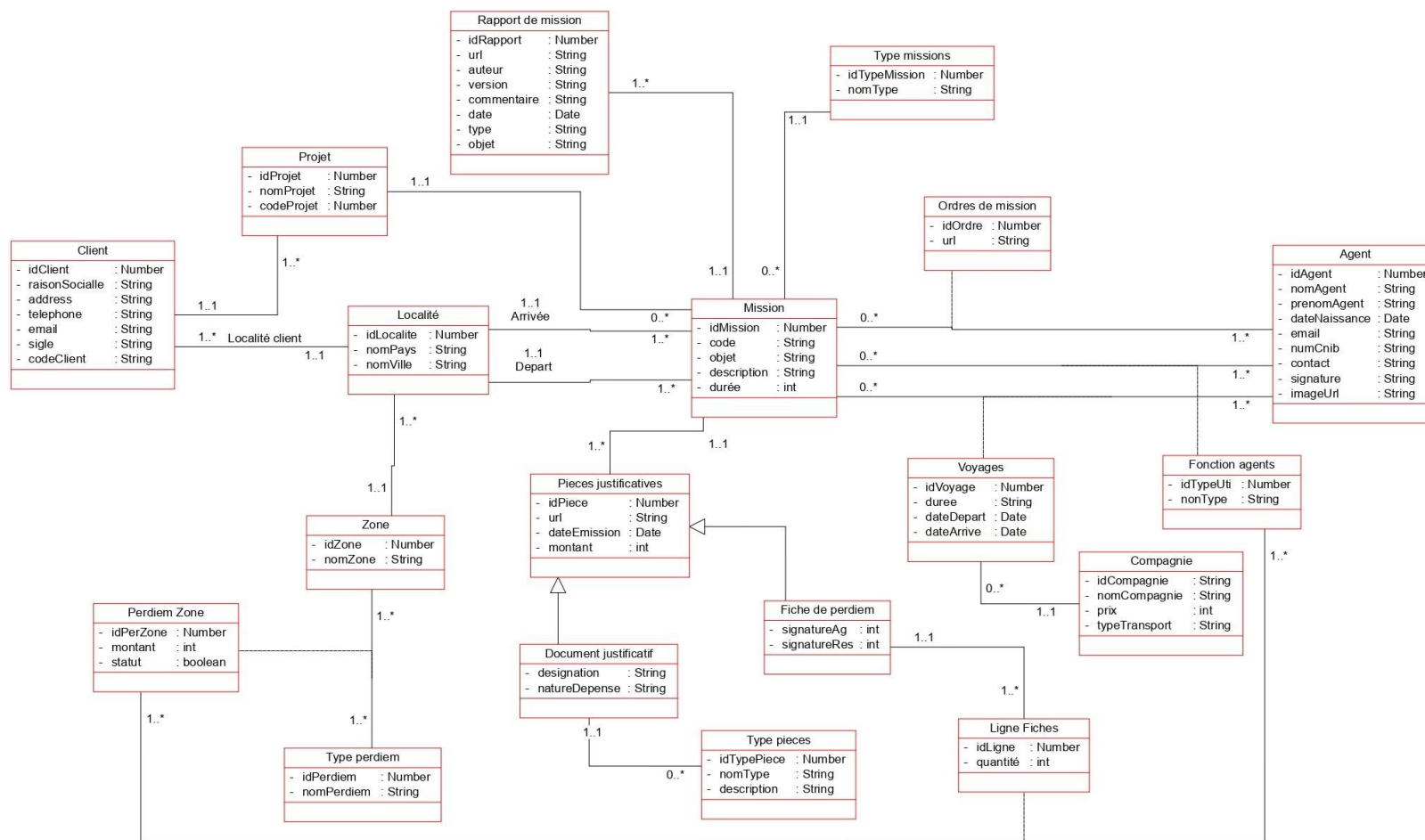


Figure 14 : diagramme de classe

3. Diagramme d'activités

Le diagramme d'activité permet de faire une représentation graphique du déroulement d'un cas d'utilisation. Les cas d'utilisation mis en exergue ici sont « Authentification », « Gérer les missions » et « Gérer fiche de perdieme », avec une vue sur les flots de données.

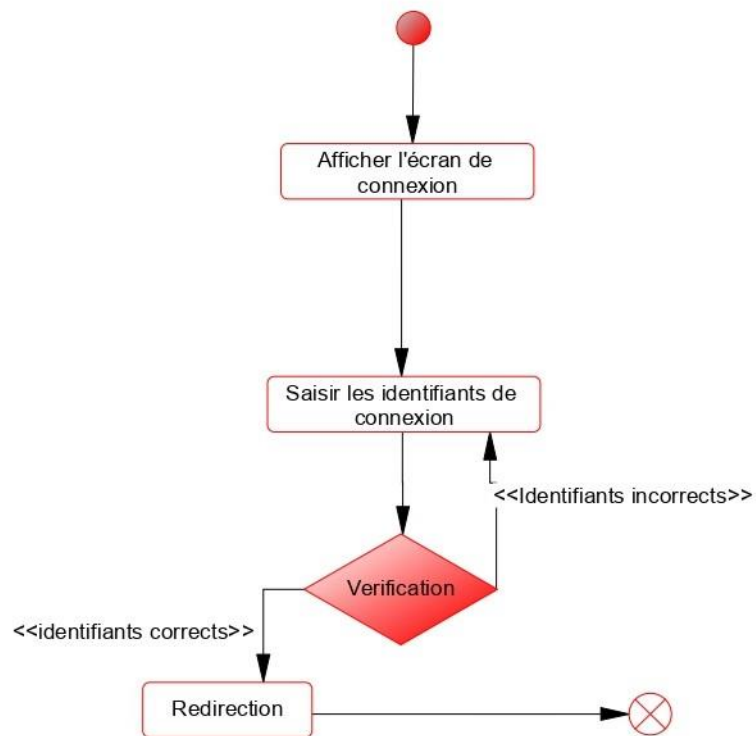


Figure 15 : diagramme d'activité du cas d'utilisation « S'authentifier »

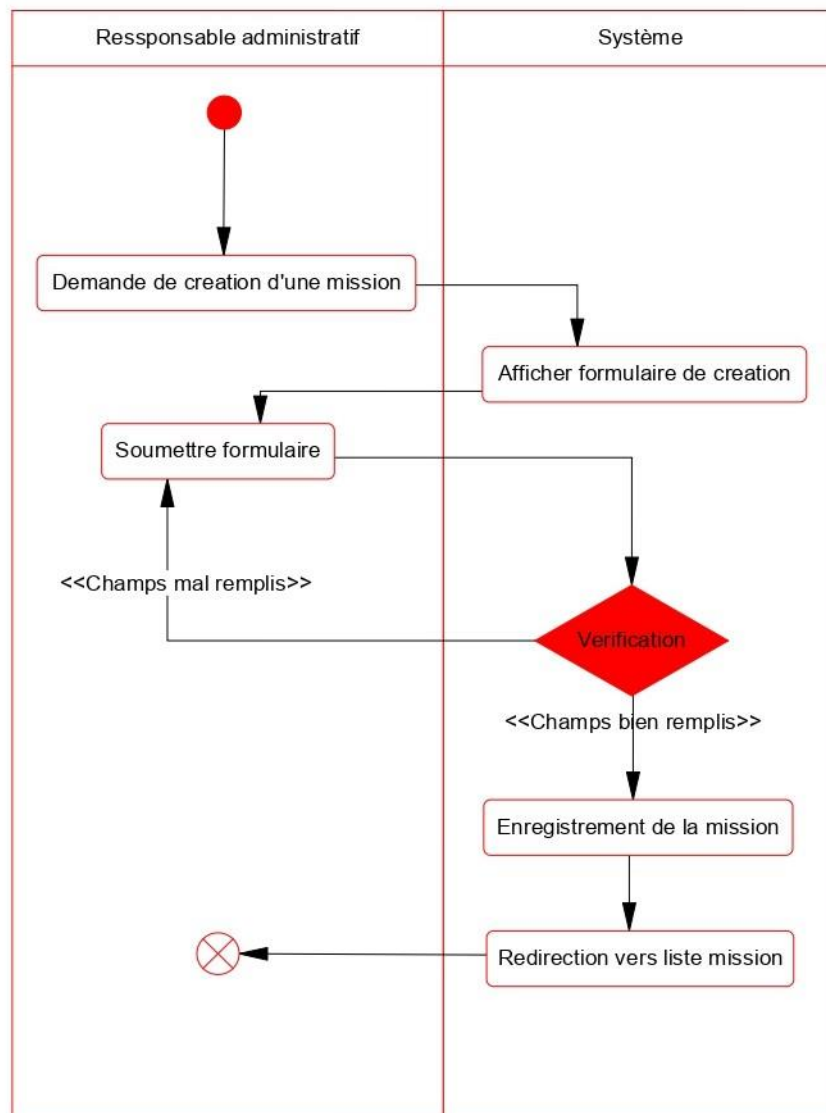


Figure 16 : diagramme d'activité du cas d'utilisation « Gérer une mission »

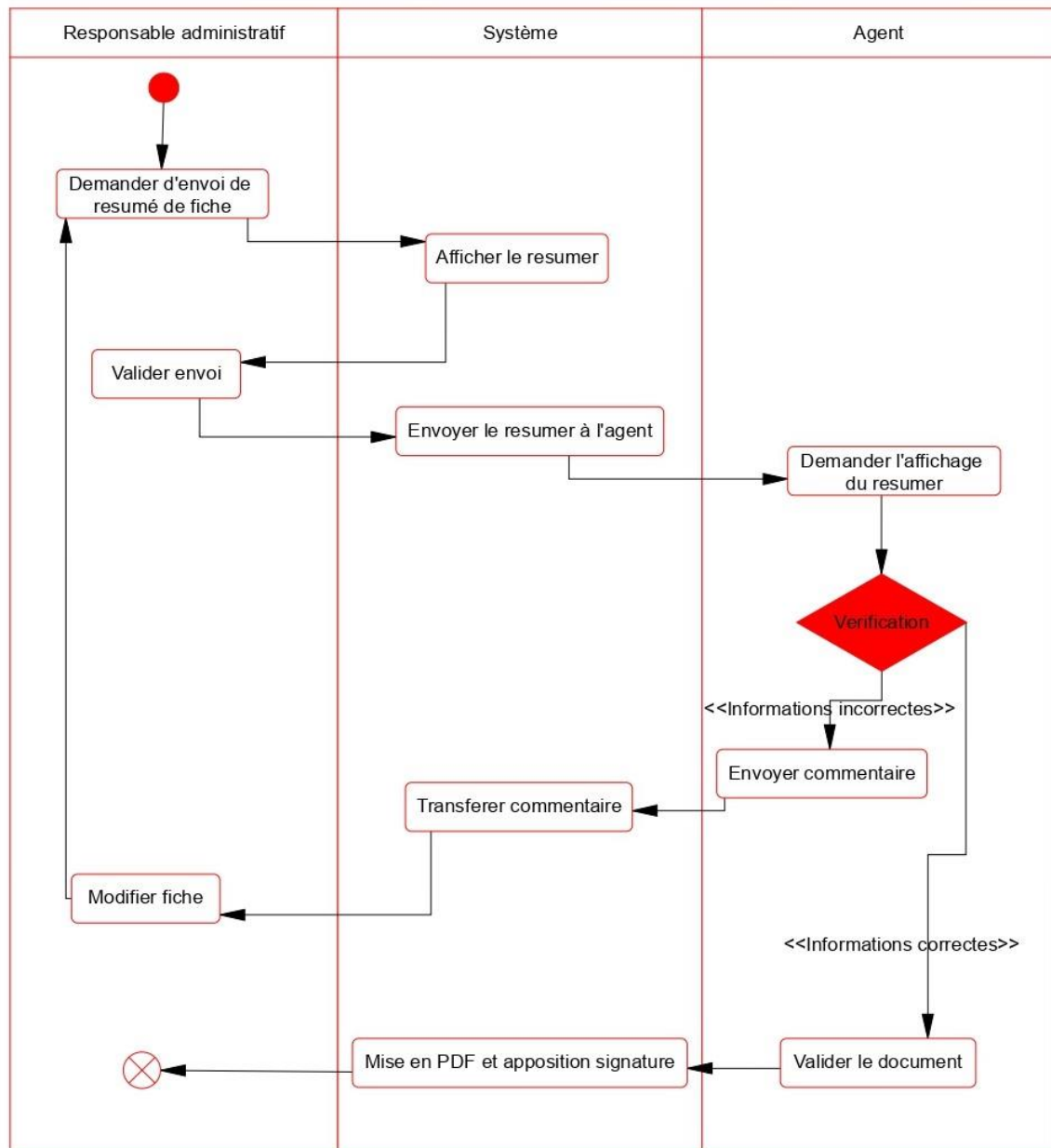


Figure 17 : diagramme d'activité du cas d'utilisation « Gérer fiche de perdieme »

4. Diagramme de déploiement

Le diagramme de déploiement décrit le déploiement physique des informations générées par le logiciel sur des composants matériels.

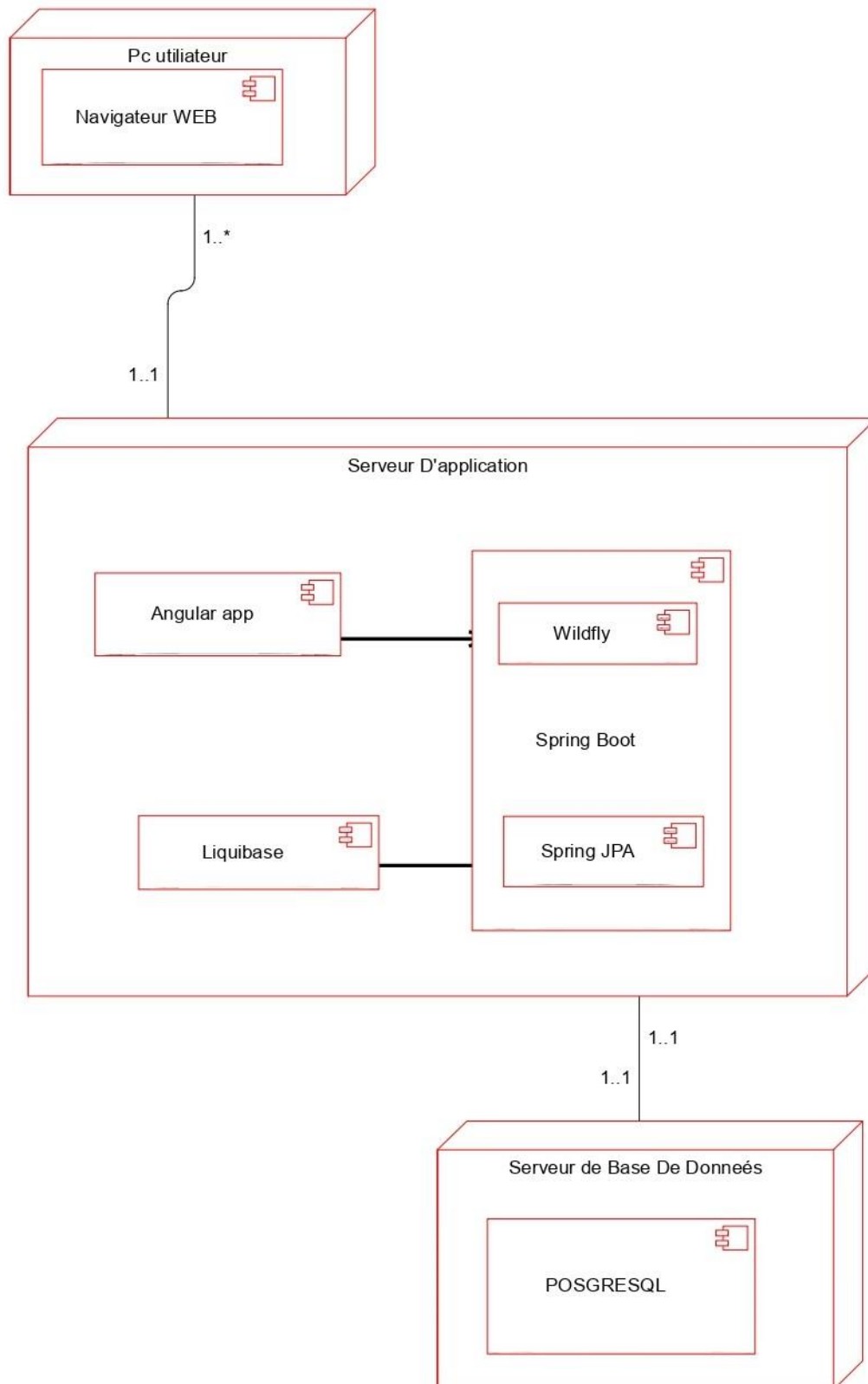


Figure 18 : diagramme de déploiement

IV. Réalisation

Dans le but de réaliser ce système nous avons utilisé plusieurs outils. Nous présenterons alors dans un premier temps les outils de réalisation, puis les tests effectués, la politique de confidentialité, ensuite nous nous attarderons sur l'estimation du coût de développement de la plateforme et pour finir nous présenterons quelques écrans de la plateforme.

1. Présentation des outils de développement

Un outil de développement est un outil qui aide les développeurs dans leurs tâches au quotidien, il en existe une multitude.

a. Choix du système de gestion de base de données (SGBD)

Dans notre domaine, il y'a plusieurs SGBD qui propose des fonctionnalités vraiment avancées.

On a entre autres :

- MySQL
- PostgreSQL
- Oracle
- Microsoft office Access

Nous avons réalisé une étude comparative qui est la suivante :

Tableau 12 : étude comparative des SGBD

SGBD	Avantages	Inconvénients
Oracle	- Oracle est conçu pour traiter de grandes quantités de données et est optimisé pour des environnements à haute charge. - Il peut facilement gérer des bases de données de grande taille avec une grande quantité d'utilisateurs simultanés.	- Oracle est une solution coûteuse, en particulier pour les grandes entreprises, car il implique des frais de licence et de maintenance élevés. - En raison de ses fonctionnalités étendues, Oracle peut être complexe à gérer et à administrer,

	- Oracle est réputé pour sa robustesse et sa fiabilité, offrant une disponibilité élevée grâce à la technologie de clustering et de redondance. - Il propose une gamme étendue de fonctionnalités avancées pour la gestion des données, telles que le partitionnement, les index avancés, la gestion des droits d'accès, etc.	nécessitant des compétences spécialisées. - Oracle peut nécessiter des ressources matérielles importantes pour fonctionner efficacement, ce qui peut être un défi pour les petites entreprises.
MySQL	- MySQL est une base de données open-source, ce qui signifie qu'elle est gratuite à utiliser et modifiable selon les besoins. - Il est relativement simple à installer, configurer et administrer, ce qui en fait un choix populaire pour les petites applications et les développeurs débutants. - MySQL offre de bonnes performances pour les applications de taille moyenne et les sites web à trafic modéré.	- MySQL peut manquer de certaines fonctionnalités avancées disponibles dans des bases de données commerciales comme Oracle. - Bien qu'il soit évolutif, il peut avoir des limitations lorsqu'il s'agit de gérer de grandes charges de travail ou des applications très complexes. - La gestion des transactions peut être moins robuste par rapport à certaines autres bases de données.

	- En tant que projet open-source populaire, MySQL bénéficie d'une large communauté d'utilisateurs et de développeurs qui fournissent un support et des ressources.	
PostgreSQL	- PostgreSQL est réputé pour sa fiabilité, sa stabilité et sa conformité aux normes SQL. - Il offre de nombreuses fonctionnalités avancées, telles que le support de JSON, les transactions ACID, les vues matérialisées, etc. - Comme MySQL, PostgreSQL est open-source, ce qui le rend gratuit et personnalisable.	- Bien que de plus en plus populaire, PostgreSQL est moins répandu que MySQL ou Oracle, ce qui peut signifier moins de ressources de support et de documentation. - La complexité des fonctionnalités avancées peut rendre son administration plus difficile pour les utilisateurs novices. - Bien que performant, il peut être moins rapide que certaines autres bases de données dans certains scénarios spécifiques.
Microsoft office Access	- Access est convivial et facile à utiliser, offrant une interface graphique intuitive pour la création de bases de données. - Il s'intègre facilement avec d'autres applications Microsoft Office, ce qui le	- Access peut avoir des limitations de taille de base de données et peut ne pas être adapté aux applications nécessitant une grande quantité de données ou un grand nombre d'utilisateurs.

	rend pratique pour les utilisateurs de cette suite logicielle. - Access est bien adapté aux petites applications de bureau ou aux applications utilisées localement.	- Il n'est pas conçu pour supporter des applications complexes ou des environnements avec de nombreux utilisateurs simultanés. - Access peut souffrir de performances médiocres lorsqu'il est utilisé pour des applications exigeantes.
--	--	---

A travers l'étude comparative que nous venons de mener, nous portons notre choix sur le SGBD PostgreSQL. En effet, PostgreSQL est gratuit et open source. Il dispose d'une grande communauté. Il est facile à déployer et à prendre en main. C'est aussi le SGBD utilisé dans l'entreprise dans laquelle nous avons effectué notre stage.

b. Serveur d'application

Nous avons également plusieurs types de serveur d'applications :

- Nginx
- GlassFish
- Wildfly
- Tomcat

Nous avons réalisé une étude comparative qui est la suivante :

Tableau 13 : étude comparative des serveurs d'application

Serveur	Avantages	Inconvénients
Nginx	- Nginx est connu pour ses performances élevées et sa capacité à gérer efficacement de nombreuses connexions simultanées, ce qui en fait un	- Nginx est principalement utilisé comme un serveur proxy ou un serveur web, mais il ne prend pas en charge directement les applications Java EE, ce qui peut

	excellent choix pour les sites à fort trafic. - Nginx utilise moins de ressources système par rapport à certains autres serveurs d'application, ce qui le rend adapté pour les environnements à ressources limitées. - Nginx peut être utilisé comme un reverse proxy pour répartir la charge sur plusieurs serveurs d'applications, ce qui améliore la scalabilité et la fiabilité. - Nginx s'intègre bien avec les conteneurs Docker et est souvent utilisé comme un proxy d'entrée pour les applications conteneurisées.	être un inconvénient pour les applications Java complètes.
GlassFish	- GlassFish est un serveur d'application Java EE complet, ce qui signifie qu'il prend en charge toutes les spécifications Java EE, y compris les EJB (Enterprise JavaBeans), les servlets, les JSP (JavaServer Pages), etc. - GlassFish est souvent considéré comme facile à utiliser et à configurer, en particulier avec les outils de développement Java EE tels que NetBeans et Eclipse. - GlassFish est bien documenté, avec une communauté active et un support Oracle, ce qui en fait un choix populaire pour les développeurs Java EE.	- GlassFish peut être plus lourd en termes de ressources système par rapport à des serveurs d'application plus légers tels que Tomcat. - Bien que performant, GlassFish peut ne pas être aussi adapté aux applications à fort trafic que certains autres serveurs d'application optimisés pour la haute performance.

Wildfly	- WildFly est un serveur d'application Java EE complet, avec une prise en charge complète des spécifications Java EE. Il est également compatible avec les dernières versions de Java EE. - WildFly est connu pour ses performances élevées et sa faible consommation de mémoire, ce qui le rend approprié pour les applications exigeantes. - WildFly propose une version adaptée aux microservices et aux conteneurs, appelée WildFly Swarm, qui permet d'embarquer uniquement les fonctionnalités nécessaires pour l'application.	- WildFly peut être plus complexe à configurer et à gérer par rapport à des serveurs d'application plus légers comme Tomcat. - Bien que la documentation de WildFly soit récente, elle peut être moins abondante que celle d'autres serveurs d'application plus populaires.
Tomcat	- Tomcat est réputé pour sa légèreté et sa faible empreinte mémoire, ce qui le rend rapide à démarrer et adapté aux applications moins gourmandes en ressources. - Tomcat est facile à installer, configurer et administrer, ce qui en fait un bon choix pour les applications web simples et les développeurs débutants. - Tomcat prend en charge les servlets Java et les JSP, ce qui permet de déployer des applications basées sur Java EE.	- En tant que serveur de servlets et de JSP, Tomcat est moins complet que GlassFish ou WildFly en termes de prise en charge complète des spécifications Java EE. - Tomcat ne prend pas en charge les EJB, ce qui peut être un inconvénient pour les applications nécessitant des fonctionnalités d'entreprise plus avancées.

A travers l'étude comparative que nous venons de présenter, nous portons notre choix sur le serveur Wildfly. En effet, Wildfly est complet et prend complètement en charge Java EE. C'est aussi un serveur extrêmement performant et qui ne consomme pas énormément en mémoire, il est également utilisé dans l'entreprise dans laquelle nous avons effectué notre stage.

Notre application s'inspire également d'une architecture de code dit monolithique qui est un modèle très répandu dans le milieu du développement en raison de sa simplicité à développer et à déployer. Bien que les pièces mobiles réelles diffèrent d'une application à une autre application, le schéma général reste le même. En général, un monolithique peut effectuer les opérations suivantes :

- il peut prendre en charge différents clients, tels que les navigateurs de bureau / mobiles et les navigateurs natifs applications de bureau/mobiles ;
- il peut exposer des API pour une consommation tierce ;
- il peut s'intégrer à d'autres applications via des services Web REST / SOAP ou files d'attente de messages ;
- il peut gérer les requêtes HTTP, exécuter la logique métier, accéder aux bases de données et échanger des données avec d'autres systèmes ;
- il peut s'exécuter sur des conteneurs d'applications Web, tels que Tomcat et JBoss ;
- il peut être mis à l'échelle verticalement en augmentant la puissance des machines sur lesquelles il fonctionne ou mise à l'échelle horizontale en ajoutant des instances supplémentaires derrière les équilibrateurs de charge.

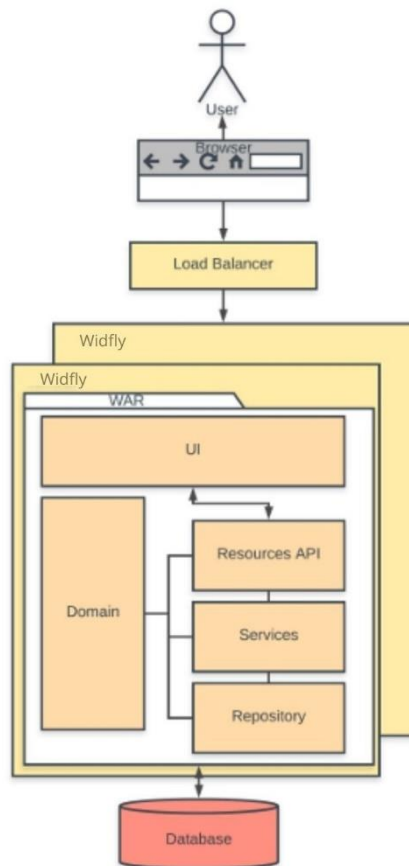


Figure 13 : architecture monolithique (Source : Full stack development with JHipster)

c. Langues

Nous avons utilisé des langages de programmations, de balisages et des langages de mise en forme pour le développement de notre système. Il s'agit des langages suivants :

- **Java 8** : Java est un langage de programmation orientée objet. (Pour les classes métiers et les entités) ;
- **TypeScript 6.11.3** : c'est un langage de programmation libre et open source développée par Microsoft qui a pour but d'améliorer et de sécuriser la production de code JavaScript;
- **HTML 5** : est un langage de marquage et de balisage servant à écrire des pages pour le World Wide Web. (Pour les pages web) ;
- **CSS 3** : le CSS permet de faire la mise en forme des pages web.

d. Plateforme de développement (Framework)

Une plateforme de développement est un ensemble de composants logiciels servant à créer les fondations ainsi que les grandes lignes d'un logiciel. On peut scinder les plateformes de

développement en deux (2) catégories ; on a des plateformes de développement back-end et des plateformes de développement front-end.

Dans la réalisation de notre application nous avons utilisé :

- **Angular 10** : est une plateforme de développement open source écrite en JavaScript qui permet la création d'applications web et particulièrement ce qu'on appelle, des « Single Page Applications ». Notre choix pour cette plateforme de développement s'explique pour sa documentation très détaillée, mais également parce qu'il propose des éléments de conception de matériaux prédéfinis tels que des éléments de navigation, des contrôles de formulaire, des tableaux de données, des mises en page et des fenêtres contextuelles;
- **Bootstrap 4** : est une infrastructure de développement frontale pour le développement avec HTML et CSS. Nous l'avons choisi car elle a l'avantage de rendre l'application responsive et fonctionne sur tous les navigateurs.
- **Spring Boot 2.1.4** : Spring Boot est un framework qui facilite le développement d'applications fondées sur Spring en offrant des outils permettant d'obtenir une application packagée en jar, totalement autonome.

e. Outils de conception

Les outils de conception représentent ici l'ensemble des outils utilisés depuis la conception jusqu'à la réalisation de la plateforme. Ce sont :

- **Power AMC version 15.1.0** : c'est un outil de conception ; il nous a donc permis de concevoir nos différents diagrammes UML.
- **Gantt Project** : Il a servi à mettre en place notre diagramme de GANTT.
- **PgAdmin** version 4 : il nous a été utile pour administrer notre base de données.
- **Git** : Git est un logiciel de gestion de versions décentralisé. Il nous a permis de gérer et archiver le code source de la plateforme.
- **GitLab** : c'est un logiciel web libre offrant un système de suivi des bugs et permettant l'intégration continue et la livraison continue. Présenté comme la plateforme des développeurs modernes, il offre la possibilité de gérer ses dépôts Git et ainsi de mieux appréhender la gestion des versions de vos codes sources.

- **Visual studio code** version 1.82.3 : est un éditeur de code qui prend en charge le débogage, la mise en évidence de la syntaxe, la complétion intelligente du code (IntelliSense4.), les snippets, la refactorisation du code et Git intégré.
- **Jhipster 6.0.1** : c'est est un générateur d'application libre et open source utilisé pour développer rapidement des applications web modernes en utilisant angular et le spring;

2. Test

Un test logiciel est un processus d'analyse d'un programme avec l'intention de détecter des anomalies dans le but de les rectifier, il permet surtout de vérifier que le logiciel répond aux attentes énoncées lors de la capture des besoins. Il en existe plusieurs mais nous nous attarderons principalement sur les tests structurels et les tests fonctionnels.

a. Test structurels

L'objectif d'un test structurel est de vérifier les fautes d'implémentation. Dans notre cas pour effectuer un test structurel on a besoin de la dépendance JUnit, du plugin testMe et d'un code de test. Nous avons testé les classes « Mission, Rapport de mission et document justificatif » ainsi que les services affilier à ces mêmes classes, la démarche des tests est décrite dans l'ANNEXE 3.

Les figure si dessous nous montre les résultats obtenus lors de nos tests structurels.

```

Terminal: C:\Window...rshell.exe x + e²
[INFO] Running DocumentJustificatifServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.824 s - in DocumentJustificatifServiceTest
[INFO] Running MissionServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.093 s - in MissionServiceTest
[INFO] Running RapportMissionServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.106 s - in RapportMissionServiceTest
[INFO] Running stage.app.config.WebConfigurerTest
2023-10-10 16:32:49.989 INFO --- [main] stage.app.config.WebConfigurer : Web application configuration, using profiles: prod
2023-10-10 16:32:50.009 INFO --- [main] stage.app.config.WebConfigurer : Web application fully configured
2023-10-10 16:32:50.076 INFO --- [main] stage.app.config.WebConfigurer : Web application configuration, using profiles: dev
2023-10-10 16:32:50.076 INFO --- [main] stage.app.config.WebConfigurer : Web application fully configured
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.603 s - in stage.app.config.WebConfigurerTest
[INFO] Running stage.app.domain.missionDomaine.DocumentJustificatifTest
[INFO] Tests run: 35, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.039 s - in stage.app.domain.missionDomaine.DocumentJustificatifTest
[INFO] Running stage.app.domain.missionDomaine.MissionTest
[INFO] Tests run: 41, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.057 s - in stage.app.domain.missionDomaine.MissionTest
[INFO] Running stage.app.domain.missionDomaine.RapportMissionTest
[INFO] Tests run: 41, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.03 s - in stage.app.domain.missionDomaine.RapportMissionTest
[INFO] Running stage.app.security.SecurityUtilsUnitTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.018 s - in stage.app.security.SecurityUtilsUnitTest
[INFO] Running stage.app.security.jwt.JWTFilterTest
2023-10-10 16:32:50.840 INFO --- [main] stage.app.security.jwt.TokenProvider : Invalid JWT signature.
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.784 s - in stage.app.security.jwt.JWTFilterTest
[INFO] Running stage.app.security.jwt.TokenProviderTest
2023-10-10 16:32:56.554 INFO --- [main] stage.app.security.jwt.TokenProvider : JWT token compact of handler are invalid.
2023-10-10 16:32:56.564 INFO --- [main] stage.app.security.jwt.TokenProvider : Expired JWT token.
2023-10-10 16:32:56.572 INFO --- [main] stage.app.security.jwt.TokenProvider : Invalid JWT signature.
2023-10-10 16:32:56.577 INFO --- [main] stage.app.security.jwt.TokenProvider : Invalid JWT signature.

```

Figure 19 : premier résultat des tests fonctionnels

```

[INFO] Running stage.app.security.SecurityUtilsUnitTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.018 s - in stage.app.security.SecurityUtilsUnitTest
[INFO] Running stage.app.security.jwt.JWTFilterTest
023-10-10 16:32:50.840 INFO --- [main] stage.app.security.jwt.TokenProvider : Invalid JWT signature.
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.784 s - in stage.app.security.jwt.JWTFilterTest
[INFO] Running stage.app.security.jwt.TokenProviderTest
023-10-10 16:32:56.554 INFO --- [main] stage.app.security.jwt.TokenProvider : JWT token compact of handler are invalid.
023-10-10 16:32:56.564 INFO --- [main] stage.app.security.jwt.TokenProvider : Expired JWT token.
023-10-10 16:32:56.572 INFO --- [main] stage.app.security.jwt.TokenProvider : Invalid JWT signature.
023-10-10 16:32:56.577 INFO --- [main] stage.app.security.jwt.TokenProvider : Invalid JWT signature.
023-10-10 16:32:56.580 INFO --- [main] stage.app.security.jwt.TokenProvider : Unsupported JWT token.
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.021 s - in stage.app.security.jwt.TokenProviderTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 150, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.3:report (post-unit-test) @ gestion-support-mission ---
[INFO] Loading execution data file D:\DossiersPerso\Stage_A2sys\PratiquePerso\GESTION SUPPORT\gestion-support-mission\target\jacoco\test.exec
[INFO] Analyzed bundle 'Gestion Support Mission' with 234 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 44.661 s
[INFO] Finished at: 2023-10-10T16:33:01Z
[INFO]

```

Figure 20 : deuxième résultat des tests fonctionnels

Ces résultats nous montrent que 150 tests ont été effectués sur les classes et les services cités précédemment, en plus de ceux-ci des tests ont été faits sur des classes de sécurisation offertes par JWT. À l'issue de ces tests nous remarquons un **build success**, et tous les tests validés. On peut donc conclure que les tests structurels sur ces ont été un succès à 100%.

b. Test fonctionnels

L'objectif d'un test fonctionnel est d'analyser le comportement d'un logiciel du point de vue de l'utilisateur. Il vise à évaluer si un logiciel ou une application répond correctement aux exigences fonctionnelles spécifiées et respecte les scénarios utilisateurs.

Pour le faire, JHipster nous fournit un scénario de tests, et pour faire ces tests, nous lançons la commande « npm test », et on a les résultats suivants :

```

Test Suites: 6 failed, 108 passed, 114 total
Tests: 3 failed, 352 passed, 355 total
Snapshots: 0 total
Time: 50.714s
Ran all test suites.
npm ERR! Test failed. See above for more details.
PS C:\Users\ASUS\Documents\stage_a2sys\app\faire_test\gestion-support-mission>

```

Figure : résultats du test fonctionnel

En observant les résultats du test on remarque que le test n'a pas été concluant à 100% et cela est due au stade auquel le test est fait. Des fonctionnalités telles que l'authentification, les routes et l'affichage des modals ont été testées.

Nous pouvons alors conclure que le test a été concluant à 97%.

3. Politique de sécurité

La politique de sécurité de l'application est le processus qui consiste à développer, ajouter et tester des fonctionnalités de sécurité au sein des applications. Elle permet d'éviter les vulnérabilités face à des menaces telles que les accès et les modifications non autorisés. La sécurité de notre plateforme représente un aspect essentiel à prendre en compte. Cette politique s'articule autour de la sensibilisation des utilisateurs, des mesures de sécurité mises en œuvre dans le processus de développement de la plateforme et des mesures à implémenter lors de sa mise en production.

➤ Sensibilisation des utilisateurs

Le facteur humain étant un élément important dans la démarche sécuritaire, il est nécessaire de fournir à nos clients des conseils pratiques pour un meilleur rendement du service fourni par la plateforme. Ainsi, l'utilisateur peut éviter certaines erreurs en adoptant les bonnes pratiques suivantes :

- éviter de communiquer ses données d'authentification à un tiers,
- ne pas céder son ordinateur à une personne sans aucune surveillance,
- se munir d'un anti-virus régulièrement mis à jour.

➤ Mesures de sécurité pendant le développement

L'accès à l'application a été protégé par un système d'authentification composé d'un « login » et d'un mot de passe. En outre, un utilisateur qui se connecte ne peut exécuter que les fonctionnalités du système que lui confère son profil. Ainsi un système d'authentification basé sur un « token JWT » a été mis en place afin de protéger l'accès aux différentes ressources de la plateforme.

➤ Mesure de sécurité à prendre pour l'exploitation

Le système de gestion de missions de A2SYS Consulting sera hébergé dans le serveur de la société, de ce fait les mesures de sécurité sont celles implémentées par l'entreprise. On peut

citer entre autres : la sécurisation de la salle des serveurs, l'attribution des droits d'accès, l'activation des systèmes de détection des intrusions.

Après la présentation de la politique de sécurité, voyons à présent à combien est estimé le coût de réalisation de notre système pour le suivi des projets de développement d'applications informatiques.

4. Estimation du coût de développement

Dans un projet informatique, l'estimation du coût de développement est très importante elle permet d'estimer l'effort à fournir et le temps que prendra le développement logiciel en fonction des ressources qui seront allouées. Il existe néanmoins plusieurs méthodes d'estimation de cout de développement, tel que :

- point de fonction,
- COCOMO,
- estimation par analogie,
- réseaux de neurones,
- arbre de régression.

Le tableau suivant nous montre une étude comparative de ces différentes méthodes :

Tableau 14 : étude comparative des méthodes d'estimation de coût

Méthodes	Types	Avantages	Inconvénients
Point de fonction	Paramétrique ou algorithmique	- Ne dépend pas du métier mais de la sémantique générique ; - Métrique objective car ne prend pas en compte les internes informatiques dont l'utilisateur final n'a ni connaissance ni utilité.	Élaborée pour des systèmes d'information de gestion pure.

COCOMO	Paramétrique ou algorithmique	- Approche permet de réduire des erreurs de l'estimation finale de l'effort ; - Ses estimations sont plus fiables si les paramètres du projet sont bien connus.	- Elle ne concerne que le développement informatique ; - Il existe une difficulté de calculer de façon fiable la taille des projets à un stade peu avancé.
Estimation par analogie	Non paramétriques (non algorithmique)	- Rapidité d'estimation du cout du projet du à l'expérience de la personne en charge de l'estimation du cout ; - Les différences entre les projets achevés et le projet proposé peuvent être identifiés et les impacts estimés.	- Ne traite pas convenablement le cas des projets logiciels décrits par des valeurs linguistiques telles que « bas », « élevé », et « complexe » ; - Elle est imprécise et non déterministe ;
Réseaux de neurones	Non paramétriques (non algorithmique)	- Permettent l'apprentissage automatique à partir des situations et des résultats précédents ; - Permettent un traitement parallèle de l'information.	- Difficultés d'explication et d'interpréter du processus ; - Problèmes de classification et de catégorisation.
Arbre de régression	Non paramétriques (non algorithmique)	- Comportement facilement compréhensible ; - Permet une modélisation des relations complexes entre les différents facteurs affectant le coût.	- Ils ne sont pas faciles à développer ; - Il implique des calculs très intensifs pour générer l'estimation.

Dans le contexte de notre projet, notre choix s'est porté sur la méthode COCOMO simplifiée (CONstructive COSt MOdel) car cette méthode permet d'estimer le coût d'un projet logiciel dans le but d'éviter les erreurs de budget et les retards de livraison. Toutes les explications sur l'utilisation de cette méthode sont présentées en Annexe 1 : La méthode COCOMO.

Nous estimons le nombre de lignes de code de notre application à sept mille (7 000). En supposant que le salaire d'un Ingénieur-informaticien est égal à 200 000 F CFA, on peut alors estimer le coût de développement comme suit :

- Effort = $2,4 * (7000/1000) 1,05 = 18,51$ Homme/Mois
- TDEV = $2,5 * \text{Effort} 0,38 = 2,5 * 18,51 0,38 = 7.57$ mois
- CDEV = Effort * X FCFA = $18,51 * 200\,000 \text{ FCFA} = 3\,702\,000 \text{ FCFA}$

En prenant en considération le coût du développement ainsi que celui du besoin en matériel informatique et logiciels, on obtient le coût total du projet tel que présenté comme suit :

Tableau 15 : coût total du projet

Désignation	Quantité	Coût unitaire (en FCFA)	Disponibilité ou coût total (en FCFA)
Ordinateur portable	1	300 000	300 000
GanttProject	1	-	Licence gratuite
Sydebase Power AMC	1	4 647 850 / an	Version d'évaluation
Postgre SQL	1	-	Licence gratuite
Visual Studio Code	1	-	Licence gratuite
Développement	-	-	3 702 000
Test logiciel	-	-	1 480 800
Total.....			5 482 800

NB : le coût du test logiciel se calcule comme suit :

CT = 40%*Développement

5. Présentation de quelques écrans de l'application



Figure 21 : écran page de destination

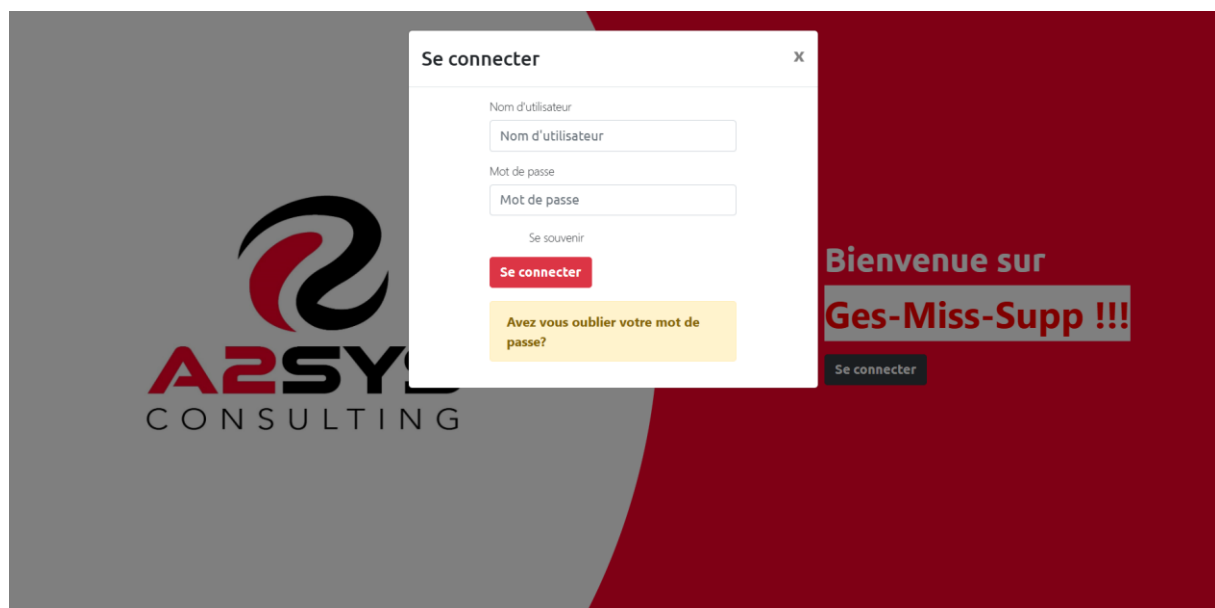


Figure 22 : écran de connexion

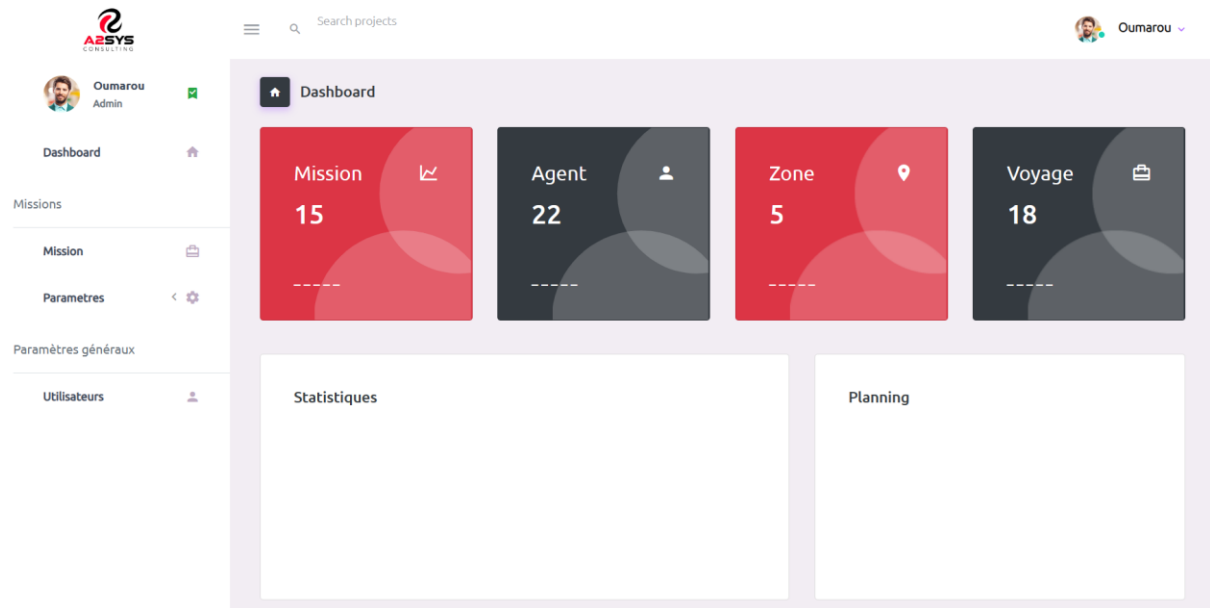


Figure 23 : écran principale

The screenshot shows the 'Créer Ou Modifier Une Mission' form. The form is organized into two columns. The left column contains fields for 'Object', 'Localité de départ', 'Date Depart', 'Projet', and 'Description'. The right column contains fields for 'Type Mission', 'Localité d'arrivée', 'Date Arrive', and 'Compagnie'. Each field is represented by a text input box. There are also two calendar icons for selecting dates. At the bottom of the form, there is a button labeled 'Ajouter des agents'.

Figure 24 : ecran d'ajout de missions

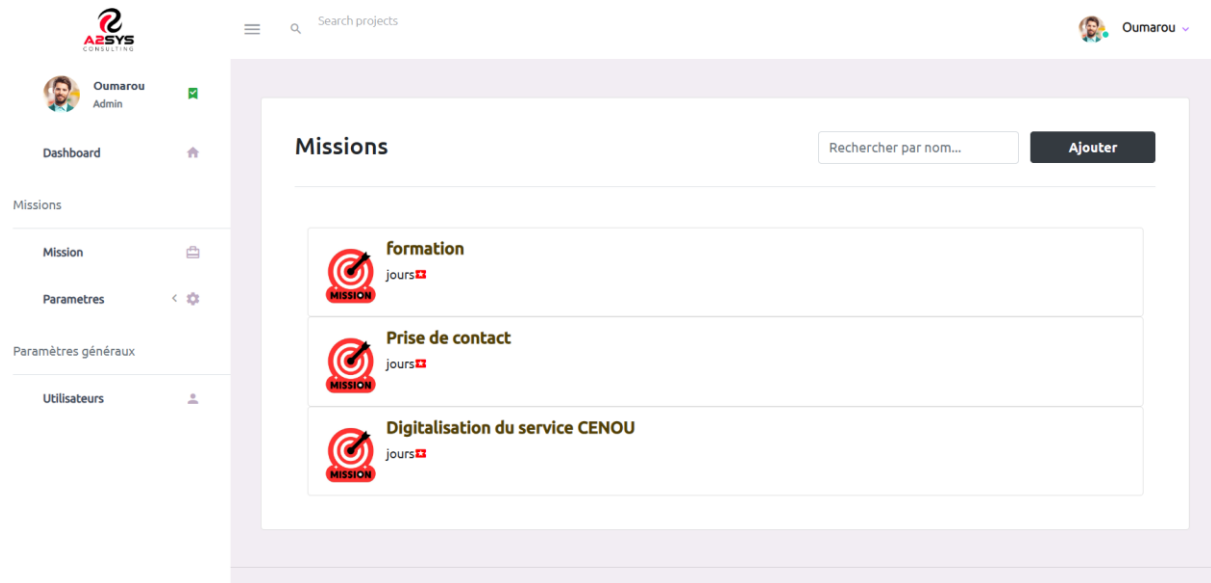


Figure : liste des missions

CHAPITRE 3 : BILAN DU STAGE

Après avoir présenté les activités menées dans le cadre de la mise en place d'une application de gestion des missions, nous faisons dans ce dernier chapitre le bilan de notre stage. Pour ce faire, nous allons d'une part présenter le déroulement du stage et les activités menées au cours de ce stage. D'autre part, nous apportons nos critiques et nos suggestions.

I. Déroulement du stage et activités menées

Notre stage s'est déroulé du 03 juillet 2023 au 03 Octobre 2023, soit une durée de trois (3) mois. Durant cette période, nous avons travaillé sur un thème d'étude suivant un planning convenu avec le Maître de stage.

1. Activités menées

Le stage s'est bien déroulé selon le chronogramme défini plus haut. Ce stage fût donc une occasion pour nous, de confronter nos acquis sur le terrain, sans oublier qu'il s'agit d'un passage qui facilitera notre insertion professionnelle. Nous avons effectué les activités suivantes :

- le développement d'une plateforme pour la gestion des missions au sein de A2SYS Consulting ;
- une initiation à l'outil Jhipster.

2. Connaissances acquises

Ce stage effectué au sein de l'entreprise A2SYS Consulting nous a été bénéfique à plusieurs niveaux.

Il nous a permis sur le plan technique de :

- mettre en pratique les connaissances académiques engrangées durant les trois (03) ans de formation à l'IBAM ;
- renforcer nos compétences en développement d'application web ;
- renforcer nos compétences en sécurisation de données ;
- renforcer nos compétences en sécurité informatique ;
- s'imprégner des réalités de la vie professionnelle ;
- d'apprendre à gérer les rapports humains au sein d'une structure.

Sur le plan professionnel, nous nous sommes frottés au monde professionnel et nous en avons tiré quelques leçons telles que :

- l'acquisition des responsabilités,
- le développement d'un réseau professionnel,
- l'obligation de nous adapter aux exigences du monde de l'entreprise.

Après avoir présenté les connaissances acquises durant notre stage, nous présentons dans la suite quelques observations et suggestions qui peuvent permettre à notre structure d'accueil et de formation d'accroître leur efficacité.

II. Observations, suggestions

Nous tenons à exprimer notre reconnaissance envers toute l'équipe d'A2SYS Consulting pour le stage qui nous a été offert. Celui-ci s'est avéré très bénéfique pour nous. Nous avons une pensée pour son accueil chaleureux dont nous avons bénéficié de la part du personnel qui, par ailleurs a mis à notre disposition des locaux agréables et conviviaux pour le travail.

Cependant nous proposons d'organiser des sessions de formation à l'arrivée des stagiaires pour faciliter la prise en main de leurs outils et de leur architecture.

CONCLUSION GENERALE

Le stage effectué au sein de A2SYS Consulting nous a permis d'approfondir nos connaissances acquises durant nos 3 années de formation en mettant en place un système de gestion de leurs missions. Dans notre processus de réalisation, nous avons opté pour la méthode 2TUP et le langage de modélisation UML à travers notamment les diagrammes de cas d'utilisation, de séquence, d'activité, de déploiement et de classes avant d'implémenter les différentes fonctionnalités identifiées.

Ce travail fut l'occasion pour nous d'approfondir nos connaissances en matière de développement d'applications avec des technologies telles que Angular, Spring Boot et de mettre en pratique nos connaissances théoriques et méthodologiques acquises au cours de notre formation à l'IBAM.

Ce stage nous a permis non seulement de nous familiariser avec le monde professionnel, mais aussi de renforcer notre capacité à travailler en équipe, de respecter des délais mais également de répondre à un problème donné en apportant une solution informatique innovante et utilisable.

En perspective, nous prévoyons de terminer l'application, et permettre de gérer les rapports de missions dans leurs évolutions jusqu'à la signature, permettre aux clients de faire des retours sur une mission effectuée mais également intégrer un système de signature numérique.

BIBLIOGRAPHIE ET WEBOGRAPHIE

Bibliographie

1. **KIEMA Prisca Barbara Wendpagnangdé (2018)**, « Mise en place d'un outil de gestion informatisée des procédures de passation des marchés publics » Rapport de fin de cycle de Licence : SIR. Ouagadougou : Ecole Supérieure des Techniques Avancées (ESTA), 113p.
 2. **OUEDRAOGO P. David (2021)**, « Mise en place d'un système de communication base sur les sms pour les professionnels et les particuliers » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 64p.
 3. **GANSONRE Rayang Newende Ange Wilfrid (2021)**, « Conception et réalisation d'une application mobile de mise en contact avec un artisan » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 52p.
 4. **KOUDA W. Ezéchias (2022)**, « Conception et réalisation d'un outil de gestion des employés de NAFANN dans un contexte de gestion de projet » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 86p.
- **Deepu K Sasidharan, Sendil Kumar N, (2020)**. *Full Stack Development with JHipster* (2e édition). Birmingham : Packt Publishing Ltd, 412p.

Webographie

1. Angular. One framework Mobile & desktop [en ligne] (page consultée plusieurs fois pendant le développement de la plateforme).
 - a. <https://angular.io/>
2. Spring. Spring makes Java simple, modern, productive, reactive, cloud-ready [en ligne] (page consultée plusieurs fois pendant le développement de la plateforme).
 - a. <https://spring.io/>

3. Documentation jhipster [en ligne] (page consultée plusieurs fois pendant le développement de la plateforme).
 - a. <https://www.jhipster.tech/>
4. Stackoverflow. Stack Overflow - Where Developers Learn, Share, and Build Careers [en ligne] (page consultée plusieurs fois pendant le développement de la plateforme).
 - a. <https://stackoverflow.com/>
5. Wikipédia. L'encyclopédie libre [en ligne] (page consultée plusieurs fois pendant le développement de la plateforme).
 - a. développement de la plateforme).
 - b. <https://fr.wikipedia.org/>
6. OpenClassroom. Site offrant des formations [en ligne] (page consultée plusieurs fois avant et pendant le développement de la plateforme).
 - a. <https://openclassrooms.com/>

TABLE DE MATIERES

DEDICACE.....	ii
REMERCIEMENTS	iii
LISTE DES SIGLES ET ABREVIATIONS	iv
LISTE DES FIGURES ET GRAPHIQUES.....	vi
LISTE DES TABLEAUX	viii
INTRODUCTION GENERALE.....	1
CHAPITRE 1 : PRESENTATION DES STRUCTURES DE FORMATION ET D'ACCUEIL	2
I. Présentation de la structure de formation (IBAM)	2
1. Historique.....	2
2. Objectif	2
3. Organisation	2
a. Organe statutaire.....	3
b. Organe d'exécution.....	4
4. Filière de formation.....	4
II. Présentation de la structure d'accueil (A2SYS consulting)	6
1. Histoire.....	6
2. Vision	7
3. Organigramme	7
4. Services	8
CHAPITRE 2 : ANALYSE ET CONCEPTION	9
I. Etude préalable.....	9
1. Etude de l'existant.....	9
a. Présentation du processus de gestion des missions	9
b. Présentation de l'existant.....	10
c. Limite de l'existant.....	10
2. Présentation du thème	11
a. Problématique.....	11
b. Résultats attendus	12
2. Méthodologie	12
a. Cycle de développement.....	12
b. Langage de modélisation	17

3. Groupe de travail.....	18
a. Groupe de pilotage.....	18
b. Groupe de projet	18
c. Groupe des utilisateurs	18
d. Planning de réalisation.....	18
II. Expression des besoins.....	19
1. Description du processus de fonctionnement attendu de l'application.....	19
2. Spécification fonctionnelle	20
a. Identification des acteurs	20
b. Identification des cas d'utilisation.....	20
c. Diagramme de cas d'utilisation	23
d. Description textuelle de certains cas d'utilisation	23
e. Diagramme de séquence	27
3. Spécifications techniques.....	31
a. Mise à disposition des conditions de travail	31
b. Architecture de développement	31
III. Conception globale.....	34
1. Dictionnaire de données.....	34
2. Diagramme de classe	36
3. Diagramme d'activités	39
4. Diagramme de déploiement	41
IV. Réalisation.....	43
1. Présentation des outils de développement	43
a. Choix du système de gestion de base de données (SGBD)	43
b. Serveur d'application.....	46
c. Langages.....	50
d. Plateforme de développement (Framework)	50
e. Outils de conception	51
2. Test.....	52
a. Test structurels	52

b. Test fonctionnels.....	53
3. Politique de sécurité.....	54
4. Estimation du coût de développement	55
5. Présentation de quelques écrans de l'application.....	58
CHAPITRE 3 : BILAN DU STAGE	61
I. Déroulement du stage et activités menées	61
1. Activités menées	61
2. Connaissances acquises	61
II. Observations, suggestions.....	62
CONCLUSION GENERALE	63
BIBLIOGRAPHIE ET WEBOGRAPHIE	I
Bibliographie.....	I
Webographie.....	I
TABLE DE MATIERES	III
ANNEXES	VI
Annexe 1 : Méthode COCOMO.....	VI
Annexe 2 : Token JWT.....	VII

ANNEXES

Annexe 1 : Méthode COCOMO

Un grand nombre de méthodes est mis à la disposition des développeurs pour estimer le coût de leurs plateformes. Notre choix se porte sur la méthode Constructive Cost Model (COCOMO) pour l'estimation du coût total du développement (CTDEV) de notre application, du fait de sa fiabilité. De plus, cette méthode permet également d'estimer le temps de développement (TDEV) du système correspondant au temps requis pour terminer le projet avec toutes les ressources disponibles. La méthode COCOMO se base principalement sur la complexité de l'application à développer qui correspond à l'un des trois (03) types suivants :

- S : Ce sont des applications simples, n'ayant que peu de cas particuliers et de contraintes. Elles sont parfaitement déterministes.
- P : Ce sont des applications intermédiaires, plus complexes que les applications de type S. Elles restent tout de même déterministes bien que le nombre de leurs cas particuliers et de tests soit plus important que pour les applications de type S.
- E : Ce sont des applications très complexes, que ce soit au niveau de leurs contraintes, comme un système temps réel, où au niveau des données saisies, comme certaines interfaces graphiques ou l'on ne peut envisager toutes les possibilités utilisateur pourrait effectuer. Elles ne sont pas déterministes.

Tableau 16 : formule de calcul COCOMO

Complexité	Effort (en Homme mois)	Temps de développement (TDEV en mois)
S	$\text{Effort} = 2,4 * \text{KLS}^{1,05}$	$\text{TDev} = 2,5 * \text{Effort}^{0,38}$
P	$\text{Effort} = 3 * \text{KLS}^{1,12}$	$\text{TDev} = 2,5 * \text{Effort}^{0,35}$
E	$\text{Effort} = 3,6 * \text{KLS}^{1,2}$	$\text{TDev} = 2,5 * \text{Effort}^{0,32}$

NB : HM est le nombre d'« homme mois » nécessaire à la réalisation du projet, et KLS est le nombre de Kilo Lignes Sources. Un homme mois correspond à 152 heures de travail effectif. Le nombre de personnes requis pour réaliser le projet dans cet intervalle de temps est donc : $N = \text{HM}/\text{TDEV}$. Étant donné que le salaire moyen d'un informaticien est de X FCFA, le coût total de développement pour ce projet est : $\text{CTDEV} = \text{HM} * X$.

Annexe 2 : Token JWT

Les JSON Web Token sont particulièrement appréciés pour les opérations d'identification. Les messages courts peuvent être chiffrés et fournissent alors des informations sûres sur l'identité de l'expéditeur et si celui-ci dispose des droits d'accès requis. Il permet l'échange sécurisé de jetons (tokens) entre plusieurs parties. Cette sécurité de l'échange se traduit par la vérification de l'intégrité et de l'authenticité des données. Elle s'effectue par l'algorithme Hash-based Message Authentication Code (HMAC) qui est un protocole utilisé pour les messages d'authentification cryptographiquement.

➤ Un jeton se compose de trois parties :

- **un en-tête (header)**, utilisé pour décrire le jeton. Il s'agit d'un objet JSON. Il est en général composé de deux parties et fournit des informations essentielles sur le token. Il contient le type de token et l'algorithme de signature et/ou de chiffrement utilisé
- **une charge utile (payload)** qui représente les informations embarquées dans le jeton. Il s'agit également d'un objet JSON. La charge utile du JSON Web Token est la partie qui contient les informations qui doivent être transmises à l'application. C'est là que sont définis certains standards qui déterminent quelles données doivent être transmises. Les informations sont fournies en paire clé/valeur, les clés sont appelées « claims » dans les JWT.
- **une signature numérique.** La signature d'un JSON Web Token est créée grâce au codage base64 de l'en-tête et de la charge utile et la méthode de signature/cryptage spécifiée. Pour que la signature fonctionne, il est nécessaire d'utiliser une clé secrète connue uniquement de l'application source. Cette signature vérifie d'une part que le message ne sera pas modifié pendant le transfert. D'autre part, dans le cas d'un jeton signé avec une clé privée, il authentifie également l'expéditeur du JWT.

➤ Fonctionnement

Les « JSON Web Token » ou JWT sont des jetons générés par un serveur lors de l'authentification d'un utilisateur sur une application Web, et qui sont ensuite transmis au client. Ils seront renvoyés avec chaque requête HTTP au serveur, ce qui lui permettra d'identifier l'utilisateur. Pour ce faire, les informations contenues dans le jeton sont signées à l'aide d'une clé privée détenue par le serveur. Quand il recevra à nouveau le jeton, le

serveur n'aura qu'à comparer la signature envoyée par le client et celle qu'il aura générée avec sa propre clé privée et à comparer les résultats. Si les signatures sont identiques, le jeton est valide.

Annexe 3 : Procédure des tests

Test structurel

➤ Objectif

L'objectif des tests structurels est de détecter les fautes d'implémentation :

- au niveau des classes le test crée des instances de pour vérifier la concordance des résultats qui sont renvoyés ;
- au niveau des services le test vérifie le bon fonctionnement des services implémenter ainsi que la justesse du code.

➤ Prérequis

Comme prérequis pour effectuer les tests structurels nous avons :

- un environnement de développement,
- un code bien implémenter,
- JUnit en tant que dépendance dans le pom.xml du projet,
- testMe comme plugin de test,
- un code de test.

➤ Principe

Le principe d'un test unitaire consiste à analyser une portion d'un programme (appelé unité ou module) afin de vérifier son bon comportement.

Afin d'aboutir aux résultats présenter plus haut :

- Importer la dépendance dans le pom.xml



```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <scope>test</scope>
</dependency>
```

Figure 25 : importation de JUnit

- Implémentation des codes de test



```
@Test
public void testSaveMission() {
    // Créez un objet MissionDTO de test
    MissionDTO missionDTO = new MissionDTO();

    // Créez un objet Mission simulé
    Mission mission = new Mission();
    mission.setId(1L);

    // Configurez les comportements simulés pour les méthodes de repository et de mapper
    when(missionMapper.toEntity(missionDTO)).thenReturn(mission);
    when(missionRepository.save(mission)).thenReturn(mission);
    when(missionMapper.toDto(mission)).thenReturn(missionDTO);

    // Appelez la méthode save
    MissionDTO savedMissionDTO = missionService.save(missionDTO);

    // Vérifiez que les méthodes mockées ont été appelées avec les bons arguments
    verify(missionMapper).toEntity(missionDTO);
    verify(missionRepository).save(mission);
    verify(missionMapper).toDto(mission);

    // Vérifiez que le résultat est correct
    assertEquals(1L, savedMissionDTO.getId());
}
```

Figure 26 : exemple d'implémentation de test pour le service « save » de la classe mission

```

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import stage.app.domain.missionDomaine.Mission;

import static org.junit.jupiter.api.Assertions.*;

public class MissionTest {

    private Mission mission1;
    private Mission mission2;

    @BeforeEach
    public void setUp() {
        // Initialisez deux objets Mission avec des valeurs identiques pour les tests
        mission1 = new Mission();
        mission1.setId(1L);
        mission1.setCode("CODE1");
        mission1.setObjet("OBJET1");
        mission1.setDescription("DESCRIPTION1");

        mission2 = new Mission();
        mission2.setId(1L);
        mission2.setCode("CODE1");
        mission2.setObjet("OBJET1");
        mission2.setDescription("DESCRIPTION1");
    }

    @Test
    public void testEquals() {
        // Vérifiez que les deux objets Mission sont égaux car ils ont la même ID
        assertEquals(mission1, mission2);
    }

    @Test
    public void testHashCode() {
        // Vérifiez que les deux objets Mission ont le même code de hachage car ils ont la même ID
        assertEquals(mission1.hashCode(), mission2.hashCode());
    }

    @Test
    public void testToString() {
        // Vérifiez que la méthode toString() renvoie une chaîne non nulle
        assertNotNull(mission1.toString());
    }
}

```

Figure 27 : exemple d'implémentation de test sur la classe mission

- Après implémentation des codes de test nous lançons la commande « mvn test » à la racine du projet et nous obtenons les résultats décrit plus haut.

Test structurel

➤ Objectif

L'objectif des tests structurels est d'analyser le comportement de l'application.

➤ Prérequis

Comme prérequis pour effectuer les tests structurels nous avons :

- un environnement de développement ;
- un code bien implémenter ;
- les classes de test générés par jhipster lors de l'implémentation avec des imports corrects.

➤ Principe

Les tests fonctionnels sont directement gérés par jhipster, celui-ci génère des codes de test au niveau front pour les faire.

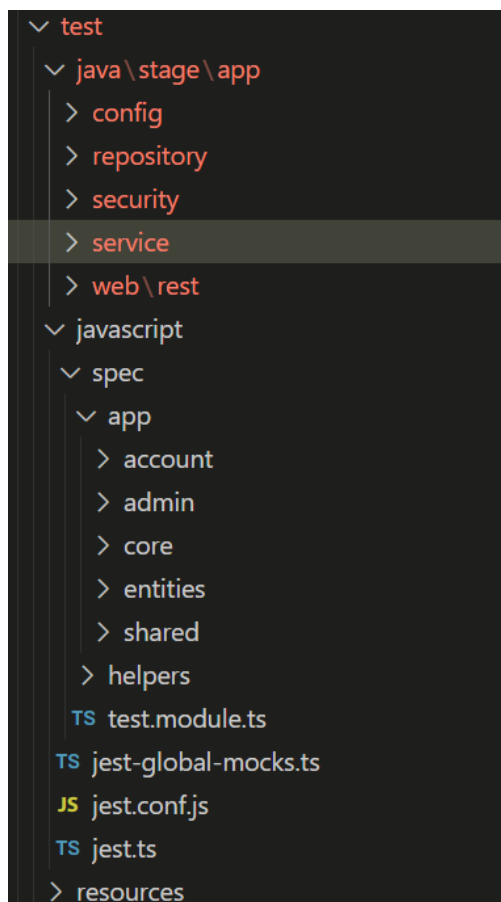


Figure 28 : détails des fichiers de test fonctionnels

Après vérification des fichiers on lance la commande « npm test » au niveau de la racine du projet pour obtenir les résultats du test.